BUILDING STATISTICAL MODELS

VISÃO COMPUTACIONAL (RECONHECIMENTO DE FACES O OBJETOS)

11

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – download model zoo YuNet 8
Figura 2 – Exemplo Faces YuNet.....11
Figura 3 – Exemplo Faces YuNet com landmarks 13
Figura 4 – Compara faces.....16
Figura 5 – Reconhece faces pessoas diferentes.....21
Figura 6 – Reconhece faces da mesma pessoa 21
Figura 7 – arquitetura vgg-face 23
Figura 8 – imagens de teste.....28
Figura 9 – gráfico de características da mesma pessoa 30
Figura 10 – gráfico de características pessoas diferentes 30
Figura 11 – Imagens da pasta faces 33
Figura 12 – Estrutura de pastas do projeto 33
Figura 13 – Exemplo de reconhecimento facial de Jeff.....37
Figura 14 – Exemplo de reconhecimento facial de Elon 38
Figura 15 – Exemplo de expressões faciais.....38
Figura 16 – Exemplo de imagens do dataset 42
Figura 17 – Resultado de treinamento do modelo.....46
Figura 18 – Resultado de treinamento do modelo.....48
Figura 19 – Resultado detecção de emoções 50
Figura 20 – exemplo imagens do dataset UTKFace 54
Figura 21 – exemplo do dataframe ajustado 56
Figura 22 – exemplo de rede multitask learning.....57
Figura 23 – Curvas de aprendizado por época 60
Figura 24 – Exemplos de predições do modelo treinado 63

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Detecta faces 10
Código-fonte 2 – Detecta objetos 12
Código-fonte 3 – Reconhece faces 19
Código-fonte 4 – vgg-face 24
Código-fonte 5 – Carrega modelo 27
Código-fonte 6 – Carrega imagens e obtém vetor de características 28
Código-fonte 7 – exibe vetor de características 29
Código-fonte 8 – calcula similaridade 31
Código-fonte 9 – Reconhecimento em vídeo 37
Código-fonte 10 – Importa dataset do google drive 41
Código-fonte 11 – Carrega os dados de treinamento, validação e teste 41
Código-fonte 11 – Carrega os dados de treinamento, validação e teste 42
Código-fonte 13 – aumento de dados 44
Código-fonte 14 – Arquitetura da rede CNN 45
Código-fonte 15 – Treinamento do modelo 45
Código-fonte 16 – Exibe curvas de aprendizagem do modelo 46
Código-fonte 17 – Exibe matriz de confusão 48
Código-fonte 18 – Emoção em tempo real 50
Código-fonte 19 – Carrega dataset UTKFace 54
Código-fonte 20 – ajusta o dataset 55
Código-fonte 21 – Criação da rede e treinamento do modelo 59
Código-fonte 22 – Função para visualizar as curvas de treinamento 59
Código-fonte 23 – Realiza predições 62

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

VISÃO COMPUTACIONAL (RECONHECIMENTO DE FACES E OBJETOS) 5

1 INTRODUÇÃO 5
1.1 Evolução da detecção de faces 5
1.2 OpenCV Model Zoo.....6
1.3 YuNet: Aplicação com o Model Zoo 7
2 Detecção de Faces vs. Reconhecimento de Faces 13
2.1 FaceRecognizer 15
2.2 Sistema de Comparação de Imagens com OpenCV Zoo e SFace 16
2.3 Treinamento de rede neural para reconhecimento facial 21
2.3.1 Explorando o modelo VGG-Face 22
2.3.2 Explorando a Representação de Imagens com VGG-Face 25
2.3.3 Explorando a Similaridade de Vetores com VGG-Face 31
2.3.3 Reconhecimento em tempo real com VGG-Face.....32
2.4 Sistema de expressões faciais.....38
2.4.1 Carregando e pré-processando o Conjunto de Dados 39
2.4.2 Treinamento da rede neural de detecção de emoções 44
2.4.3 Detecção de emoções em tempo real.....49
2.5 Sistema de predição de idade e sexo 51
2.5.1 Conhecendo e preparando os Dados.....53
2.5.2 Definindo a arquitetura do modelo 56
2.5.3 Detecção de emoções em tempo real.....61

3 HANDS ON: PRATICANDO O RECONHECIMENTO DE PESSOAS.....64
3.1 Aquecimento 64
6.2 Seu desafio 64
6.3 Dicas de solução.....65

REFERÊNCIAS 66

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

VISÃO COMPUTACIONAL (RECONHECIMENTO DE FACES E OBJETOS)

1 INTRODUÇÃO

Na área da visão computacional, poucas problemáticas são tão interessantes e cativantes quanto o reconhecimento de rosto e objetos. Neste capítulo, embarcamos em uma exploração do fascinante mundo da decifração de rostos humanos. Vamos entender a diferença entre detectar e reconhecer faces em imagens, distinguindo suas características e identificando os indivíduos aos quais pertencem.

1.1 Evolução da detecção de faces

As técnicas clássicas de detecção de faces, como as baseadas em Haar cascade, desempenharam um papel crucial no início do desenvolvimento da visão computacional. Introduzidas por Viola e Jones em 2001, essas técnicas utilizavam um classificador em cascata de Haar-features para detectar rostos em imagens. Apesar de serem rápidas e relativamente eficazes para a época, essas técnicas apresentavam limitações significativas.

Um dos principais problemas com o método Haar cascade era a taxa de falsos positivos. Ou seja, muitas vezes, objetos ou áreas na imagem que não eram rostos eram erroneamente detectados como tal. Além disso, a técnica tinha dificuldades em detectar rostos em condições variadas de iluminação, ângulos diferentes ou com obstruções parciais, o que limitava sua aplicabilidade em cenários mais realistas e complexos.

Com o avanço da tecnologia e a crescente demanda por soluções mais robustas, surgiram técnicas mais modernas que superaram essas limitações. Uma dessas técnicas é o YuNet, uma rede neural convolucional baseada na arquitetura MobileNetV2, projetada especificamente para a detecção de faces. YuNet se destaca por sua capacidade de detectar rostos com maior precisão e robustez, mesmo em condições desafiadoras.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

YuNet é um detector de faces leve (tiny) e extremamente rápido, com nível de precisão de milissegundos. A sua arquitetura foi projetada para ser rápida e eficiente, mantendo uma alta precisão. Comparado a outros detectores de faces, como o YOLOv4, o YuNet mostrou ser mais preciso. Aqui, estão algumas das principais características do YuNet:

- Modelo compacto: com apenas 75.856 parâmetros;
- Rápido e eficiente: velocidade de inferência de 1,6 milissegundos por frame em uma CPU Intel i7-12700K;
- Alta precisão: mAP de 81,1% no conjunto de validação hard do WIDER FACE;
- Versátil: pode ser usado em uma variedade de dispositivos, incluindo telefones móveis, sistemas embarcados e dispositivos edge.

YuNet é uma arquitetura que deve ser considera em projetos reais para a detecção de faces, ideal para aplicações onde a velocidade e a precisão são essenciais, como reconhecimento facial em tempo real e vigilância por vídeo.

Você deve estar ansioso para usar o Yunet, mas antes de sair criando nossos códigos, vamos conhecer model Zoo, que irá facilitar muito nossas tarefas de visão computacional.

Para seu conhecimento: a detecção de faces é tipicamente o primeiro passo em muitos sistemas de reconhecimento facial porque é necessário localizar os rostos antes de poder reconhecê-los.

1.2 OpenCV Model Zoo

O **OpenCV Model Zoo** é um recurso valioso para trabalhar com projetos de visão computacional. Trata-se de uma coleção de modelos pré-treinados e pesos de modelos que podem ser usados com o OpenCV. O Model Zoo contém uma gama de modelos projetados para diferentes tarefas de visão computacional, incluindo detecção de objetos, classificação de imagens, **reconhecimento facial**, detecção de texto e muito mais. Esses modelos são treinados em grandes conjuntos de dados e muitas vezes são os mais avançados em termos de precisão e desempenho.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Usar modelos pré-treinados do Model Zoo é simples. O OpenCV fornece APIs convenientes para carregar esses modelos e aplicá-los a imagens ou vídeos. O Model Zoo é um esforço colaborativo, e as contribuições da comunidade de visão computacional são bem-vindas. Os desenvolvedores podem encontrar modelos treinados em vários conjuntos de dados e para casos de uso específicos, o que pode ser extremamente valioso para seus projetos. Ao aproveitar os modelos pré-treinados do Model Zoo, os desenvolvedores podem reduzir significativamente o tempo e os recursos computacionais necessários para treinar modelos de aprendizado profundo do zero. Nós, como desenvolvedores, frequentemente usamos esses modelos pré-treinados como ponto de partida para seus próprios experimentos ou projetos. Isso permite uma prototipagem e experimentação rápidas antes de se comprometerem a treinar modelos personalizados. Seja para reconhecimento de objetos, segmentação de imagens ou qualquer outra tarefa relacionada à visão, o Model Zoo provavelmente tem um modelo pré-treinado que pode acelerar o trabalho.

Dica: Saiba mais sobre o Model Zoo acessando o site oficial onde você pode encontrar e baixar os modelos pré-treinados. Sendo contribuições open-source, os modelos são frequentemente atualizados com as implementações mais modernas.

https://github.com/opencv/opencv_zoo

1.3 YuNet: Aplicação com o Model Zoo

Agora que fomos apresentados ao Model Zoo, vamos utilizá-lo para fazer a detecção de faces com o YuNet.

Vamos dividir essa prática em 3 passos:

- Baixar o Modelo YuNet do Model Zoo;
- Carregar o Modelo YuNet no OpenCV;
- Realizar as predições do modelo.

Vamos realizar o passo a passo de cada uma das etapas.

Passo 1: Baixar o modelo YuNet do Model Zoo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O YuNet está disponível no OpenCV Model Zoo. Você pode baixar o modelo YuNet (.onnx) diretamente do GitHub, no link a seguir: https://github.com/opencv/opencv_zoo/tree/main/models/face_detection_yunet

Baixe o arquivo que possua a extensão `xxx.onnx` do diretório, onde xxx representa o prefixo do arquivo, e lembre-se que os modelos são atualizados, logo, algumas mudanças nesse nome do arquivo podem ocorrer, e “onnx” (Open Neural Network Exchange) trata-se de um ecossistema de inteligência artificial de código aberto que estabelece padrões para representar modelos de aprendizado de máquina. A figura a seguir exibe como realizar o download do modelo YuNet direto do repositório github, basta fazer o download clicando no ícone em destaque vermelho.

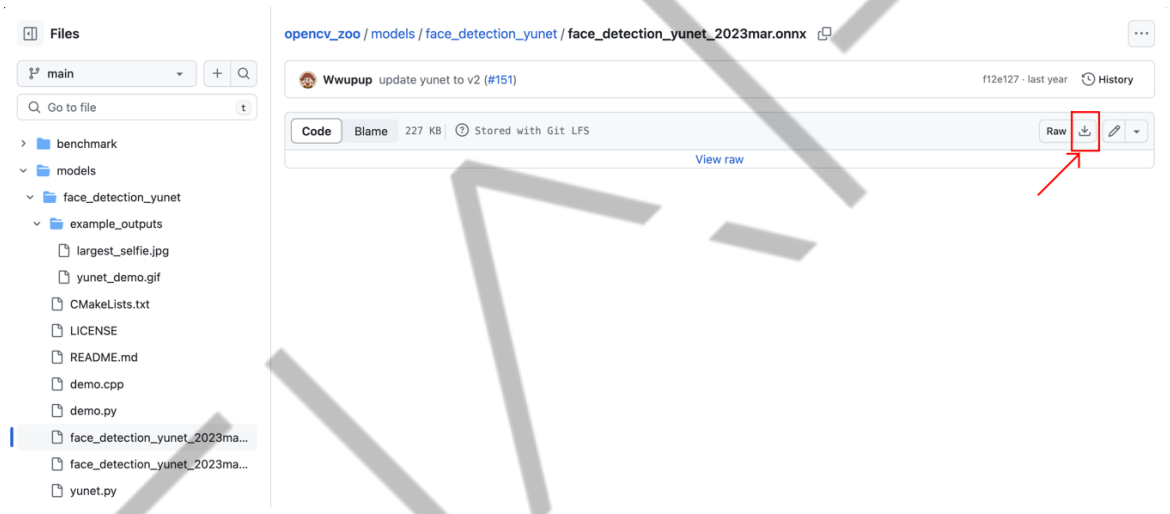

Figura 1 – download model zoo YuNet
Fonte: autor (2024)

Passo 2: Carregar o Modelo YuNet no OpenCV

Vamos utilizar as funcionalidades de detecção de rostos da biblioteca OpenCV para realizar predições com o modelo YuNet. Desenvolveremos um script Python que configura e carrega diretamente o modelo YuNet usando a interface `cv2.FaceDetectorYN`. Esse script cuidará do pré-processamento das imagens conforme as necessidades do modelo para detecção de rostos em imagens ou vídeo em tempo real.

```
import cv2
import numpy as np


def desenha_face(input, faces):
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    if faces[1] is not None:
        for face in faces[1]:
            x = int(face[0])
            y = int(face[1])
            w = int(face[2])
            h = int(face[3])
            confiance = round(face[-1], 2)

            # desenhar o retângulo em torno do rosto
            cv2.rectangle(input, (x, y), (x + w, y + h), (255, 0,
255), 4) # face box
            cv2.putText(input, str(confiance), (x,y),
cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 0, 0), 4, cv2.LINE_AA)
    return input

def config_detection(img):
     # Inicializa FaceDetectorYN com parâmetros
    nms_threshold = 0.3
    score_threshold = 0.9
    yunet_shape = (img.shape[1], img.shape[0])
    topk = 5000
    # Inicializa o detector de rosto
    detector =
cv2.FaceDetectorYN.create("face_detection_yunet_2023mar.onnx", "",
yunet_shape, score_threshold, nms_threshold, topk)
    return detector


def main_image():

    # Carrega a imagem
    imgpath = "img.jpg"
    img = cv2.imread(imgpath)

    # Configura detecção do YuNet
    detector = config_detection(img)

    # Detecta face
    face = detector.detect(img)

    # Desenha o rosto na imagem
    desenha_face(img, face)

    cv2.imshow("faces", img)
    cv2.waitKey(0)

def main_video():

    # Inicializa a captura de vídeo
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    cap = cv2.VideoCapture(0)  # Substitua 0 por um caminho de arquivo
se necessário, por exemplo "/video/meu_video.mp4"

    # Obtém a largura e altura do frame
    ret, frame = cap.read()
    if ret:
    # Configura o detector com base no frame capturado
        detector = config_detection(frame)

    while True:
        ret, frame = cap.read()

        if not ret:
            break  # Sai do loop se não conseguir ler o frame

        # Detecta rostos no frame
        faces = detector.detect(frame)

        # Desenha os rostos detectados no frame
        frame = desenha_face(frame, faces)

        # Mostra o frame com detecções
        cv2.imshow("face detections", frame)

        # Encerra o loop se a tecla 'q' for pressionada
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == '__main__':

    # main_image() # Descomentar para rodar uma imagem
    main_video()  # Descomentar para rodar o vídeo
```

Código-fonte 1 – Detecta faces
Fonte: autor (2024)

É esperado um resultado similar ao da figura a seguir com a deteção dos rostos presentes na imagem, o resultado é semelhante se for utilizado para captura de vídeo, mas nesse caso, será capturada a imagem da webcam.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 2 – Exemplo Faces YuNet
Fonte: autor (2024)

Passo 3: Realizar as predições do modelo

Já conseguimos realizar algumas predições, mas podemos adaptar o código e utilizar a YuNet para capturar alguns landsmarks da face. Os Landmarks, também conhecidos como pontos de referência faciais ou pontos-chave faciais, são pontos específicos no rosto de uma pessoa que correspondem a características anatômicas distintas. Esses pontos de referência são frequentemente usados como pontos de referência chave para várias tarefas de visão computacional, incluindo reconhecimento facial.

Vamos modificar apenas a nossa função desenha_face() para incluir os landsmarks.

```
def desenha_face(input, faces):
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    if faces[1] is not None:
        for face in faces[1]:
            # Converte as coordenadas para intero
            coordinates = face[:-1].astype(np.int32)
            x = coordinates[0]
            y = coordinates[1]
            w = coordinates[2]
            h = coordinates[3]
            confiance = round(face[-1], 2)

            # Desenhar o retângulo em torno do rosto
            cv2.rectangle(input, (x, y), (x + w, y + h), (255, 0,
255), 4) # Caixa de detecção de rosto
            cv2.putText(input, str(confiance), (x, y),
cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 0, 0), 4, cv2.LINE_AA)

            # coordenadas do rosto e pontos de referência
            righteye_x = coordinates[4]
            righteye_y = coordinates[5]

            lefteye_x = coordinates[6]
            lefteye_y = coordinates[7]

            nosetip_x = coordinates[8]
            nosetip_y = coordinates[9]

            mouth_rightcorner_x = coordinates[10]
            mouth_rightcorner_y = coordinates[11]

            mouth_leftcorner_x = coordinates[12]
            mouth_leftcorner_y = coordinates[13]

            cv2.line(input, (righteye_x, righteye_y), (lefteye_x,
lefteye_y), (255, 0, 255), 8, lineType=cv2.FILLED) # olhos
            cv2.line(input, (mouth_leftcorner_x, mouth_leftcorner_y),
(mouth_rightcorner_x, mouth_rightcorner_y), (255, 0, 0), 8,
lineType=cv2.LINE_4) # boca
            cv2.circle(input, (nosetip_x, nosetip_y), 8, (0, 127,
255), -1) # nariz

    return input
```

Código-fonte 2 – Detecta objetos
Fonte: autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Com a modificação da nossa função `desenha_face`, adicionamos os landmarks da face que representam as coordenadas dos olhos, nariz e boca. Isso permite uma visualização mais detalhada e informativa ao detectar rostos em imagens ou vídeos. Na figura a seguir, cada landmark está destacando pontos críticos que são importantes para aplicações como reconhecimento facial, análise de expressões ou mesmo para melhorias em aplicações de realidade aumentada.

Figura 3 – Exemplo Faces YuNet com landmarks
Fonte: autor (2024)

2 Detecção de Faces vs. Reconhecimento de Faces

A detecção e o reconhecimento facial são tarefas distintas dentro do campo da visão computacional, cada uma com propósitos específicos. Detecção facial consiste em identificar e localizar faces dentro de uma imagem, enquanto o reconhecimento facial avança um passo além, determinando a identidade das faces detectadas. A detecção facial é geralmente o primeiro passo nos sistemas de reconhecimento facial, pois é necessário localizar as faces antes de reconhecê-las. Compreender essas

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

distinções é crucial ao trabalhar com aplicações que envolvem faces, seja para segurança, entretenimento ou outros fins.

O quadro a seguir resume algumas diferenças entre as tarefas de detecção e reconhecimento facial como objetivo, o tipo de saída esperada, casos de usos e a complexidade envolvida.

Aspecto	**Detecção Facial**	**Reconhecimento Facial**
Tarefa	Identificar a presença de faces em uma imagem.	Identificar ou verificar a identidade de uma pessoa com base em suas características faciais.
Objetivo	Determinar se há uma face nos dados de entrada e, em caso afirmativo, fornecer as coordenadas ou caixa delimitadora em torno das faces detectadas.	Determinar quem é a pessoa comparando suas características faciais com um banco de dados de indivíduos conhecidos.
Saída	Uma caixa delimitadora que envolve as faces detectadas.	A identidade das faces detectadas ou uma determinação de correspondência com uma pessoa conhecida.
Casos de Uso	Utilizado em várias aplicações, como autofoco em câmeras digitais, contagem de pessoas em uma multidão, rastreamento de faces em vídeos para segurança.	Usado em aplicações como desbloqueio de smartphones com reconhecimento facial, verificação de identidade em aeroportos, sistemas de vigilância para identificação de pessoas de interesse.
Complexidade	Relativamente mais simples comparado ao reconhecimento facial.	Tarefa mais complexa, pois envolve criar uma representação de características da face e então compará-la a um banco de dados

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

		de faces conhecidas para fazer uma identificação.

A OpenCV oferece suporte a uma ampla variedade de algoritmos de reconhecimento facial. Entre eles, está o `FaceRecognizer`, que proporciona grande flexibilidade para suportar diferentes algoritmos de reconhecimento facial. Vamos entender e aplicar o módulo FaceRecognizer na próxima seção.

2.1 FaceRecognizer

Todos os modelos de reconhecimento facial no OpenCV são derivados da classe base abstrata FaceRecognizer. Cada FaceRecognizer é tratado como um algoritmo, permitindo o acesso ao seu funcionamento interno por meio de operações simples de obtenção e configuração, quando permitido pela implementação. Essa classe fornece três métodos principais para reconhecimento facial: Eigenfaces, Fisherfaces e Local Binary Patterns Histograms (LBPH). Cada um desses métodos tem suas próprias vantagens e desvantagens, dependendo do contexto em que são usados.

- Eigenfaces: o algoritmo Eigenfaces utiliza Análise de Componentes Principais (PCA) para reduzir a dimensionalidade das imagens faciais e encontrar a direção principal da variação entre as faces. A ideia é projetar as imagens em um subespaço de menor dimensionalidade, onde as faces possam ser comparadas;
- Fisherfaces: o algoritmo Fisherfaces utiliza Análise de Discriminantes Lineares (LDA) para maximizar a separabilidade entre as classes (pessoas) enquanto reduz a variabilidade dentro das classes. Isso resulta em um modelo mais robusto para condições de iluminação variadas;
- LBPH (Local Binary Patterns Histograms): o algoritmo LBPH codifica as relações locais entre os pixels em padrões binários, criando histogramas desses padrões. É robusto a variações de iluminação e é geralmente mais simples de implementar.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Além desses módulos amplamente conhecidos, podemos utilizar módulos baseados em deep learning, como SFace.

2.2 Sistema de Comparação de Imagens com OpenCV Zoo e SFace

Vamos utilizar o OpenCV Zoo com o modelo SFace para realizar reconhecimento facial, combinado ao Sface. Dessa forma, vamos poder criar um sistema de reconhecimento facial em tempo real, que funciona com qualquer sistema operacional, assim como demonstra a imagem a seguir, onde, à esquerda, temos as imagens que serão comparadas com a imagem de referência e o resultado da comparação.

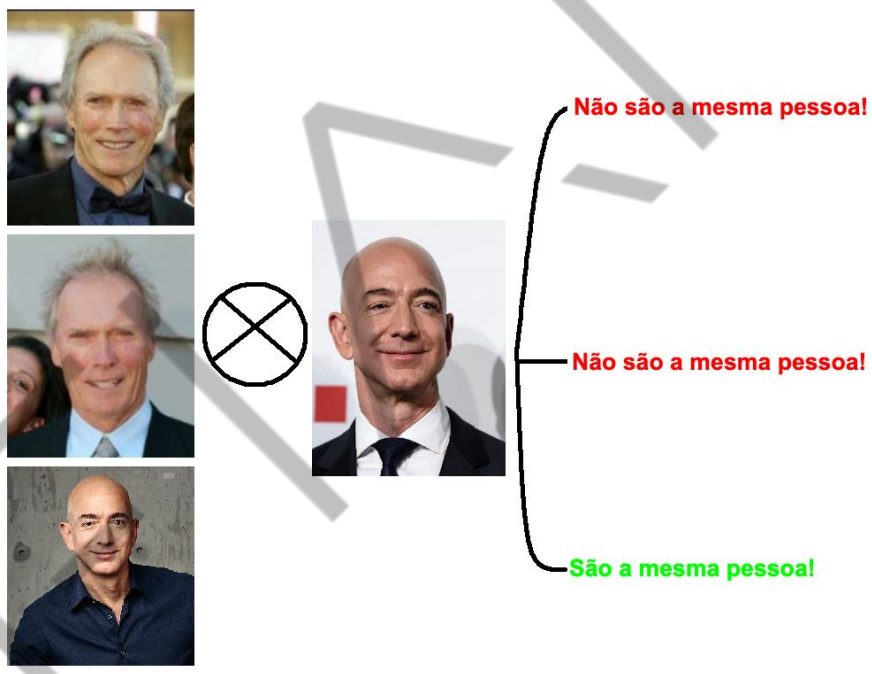

Figura 4 – Compara faces
Fonte: autor (2024)

Agora que já entendemos como sistema irá funcionar, vamos começar a criar nosso sistema. Comece acessando o Opencv model Zoo e faça o download dos modelos:

- Yunet.onnx (para detecção de face);

- Sface.onnx (para reconhecimento de face).

Além disso, selecione algumas imagens da mesma pessoa e de pessoas diferentes para testar o funcionamento do reconhecedor de faces.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
     import numpy as np
     import cv2

def config_detection(img):
    # Inicializa FaceDetectorYN com parâmetros
    nms_threshold = 0.3
    score_threshold = 0.9
    yunet_shape = (img.shape[1], img.shape[0])
    topk = 5000
    # Inicializa o detector de rosto
    detector =
cv2.FaceDetectorYN.create("face_detection_yunet_2023mar.onnx", "",
yunet_shape, score_threshold, nms_threshold, topk)
    return detector

def detect_faces(detector, img):
    detector.setInputSize((img.shape[1], img.shape[0]))
    faces_and_landmarks = detector.detect(img)
    return faces_and_landmarks

def align_and_extract_features(recognizer, img, faces_and_landmarks):
    face_align = recognizer.alignCrop(img, faces_and_landmarks[1][0])
    facial_features = recognizer.feature(face_align)
    return facial_features

def calculate_similarity(recognizer, features1, features2,
cosine_threshold, l2_threshold):
    cosine_score = recognizer.match(features1, features2,
cv2.FaceRecognizerSF_FR_COSINE)
    l2_score = recognizer.match(features1, features2,
cv2.FaceRecognizerSF_FR_NORM_L2)
    return cosine_score, l2_score

def draw_faces(img, faces_and_landmarks, color, text):
    if faces_and_landmarks is not None:
        for face in faces_and_landmarks[1]:
            bbox = face[:4].astype(int)
            cv2.rectangle(img, (bbox[0], bbox[1]), (bbox[0]+bbox[2],
bbox[1]+bbox[3]), color, 2)
            cv2.putText(img, text, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
1, color, 2)
    return img

def main():
    # Carrega as imagens
    img1path = "img20.png"
    img2path = "img21.png"
    img1 = cv2.imread(img1path)
    img2 = cv2.imread(img2path)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    # Configura detecção do YuNet
    detector1 = config_detection(img1)
    detector2 = config_detection(img2)

    # Detecta faces
    faces_and_landmarks1 = detect_faces(detector1, img1)
    faces_and_landmarks2 = detect_faces(detector2, img2)

    # Inicializa FaceRecognizerSF
    recognizer =
cv2.FaceRecognizerSF.create("face_recognition_sface_2021dec.onnx", "")

    # Alinha e extrai características faciais
    facial_features1 = align_and_extract_features(recognizer, img1,
faces_and_landmarks1)
    facial_features2 = align_and_extract_features(recognizer, img2,
faces_and_landmarks2)

    # Define os limites de similaridade
    cosine_similarity_threshold = 0.363 # quanto mais próximo de 1,
mais similar
    l2_similarity_threshold = 1.128 # quanto mais próximo de 0, mais
similar

    # Calcula as similaridades
    cosine_score, l2_score = calculate_similarity(recognizer,
facial_features1, facial_features2, cosine_similarity_threshold,
l2_similarity_threshold)

    # Determina se as imagens pertencem à mesma pessoa
    print("Similaridade: Coseno - {}, L2 - {}".format(cosine_score,
l2_score))
    if (cosine_score >= cosine_similarity_threshold) or (l2_score <=
l2_similarity_threshold):
        print("São a mesma pessoa")
        color = (0, 255, 0)
        text = "Sao a mesma pessoa"
    else:
        print("Não são a mesma pessoa")
        color = (0, 0, 255)
        text = "Nao sao a mesma pessoa"


    # Exibe as imagens com as detecções de rosto
    #img1_with_faces = draw_faces(img1, faces_and_landmarks1, color,
text)
    img2_with_faces = draw_faces(img2, faces_and_landmarks2, color,
text)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    # Mostra as imagens
    cv2.imshow("Imagem Referência", img1)
    cv2.imshow("Imagem Teste", img2_with_faces)

    # Aguarda uma tecla para fechar as janelas
    cv2.waitKey(0)
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main()
```

Código-fonte 3 – Reconhece faces
Fonte: autor (2024)

Vamos compreender em detalhes o funcionamento do nosso código. A função principal é a main, que realiza as seguintes tarefas:

- **Carregamento das Imagens**: as imagens de referência e de teste são carregadas;
- **Configuração do Detector**: o detector YuNet é configurado para cada imagem por meio da função config_detection;
- **Detecção de Rostos**: a função detect_faces é responsável por detectar as faces nas duas imagens;
- **Inicialização do Reconhecedor Facial**: o modelo de reconhecimento facial Sface é carregado com a função da openCV FaceRecognizerSF;
- **Extração de Características Faciais**: as características faciais são extraídas por meio da função align_and_extract_features, que primeiro faz o crop da face, excluindo regiões que não são de interesse; o método feature extrai o vetor de características faciais. Isso é importante, pois esse vetor de características pode então ser comparado com vetores de outras imagens para determinar se os rostos são da mesma pessoa;
- **Cálculo da Similaridade**: para determinar se dois rostos pertencem à mesma pessoa, comparamos os vetores de características faciais extraídos de cada rosto. Essas comparações são feitas usando métricas de similaridade que quantificam o quão próximos ou diferentes os vetores são. A openCV disponibiliza duas dessas métricas: **cosseno** e **L2**. Vamos entender uma intuição sobre cada métrica.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Similaridade Cosseno**: a similaridade cosseno mede o ângulo entre dois vetores. Valores mais próximos de 1 indicam que os vetores são mais semelhantes. Como indica a equação a seguir, onde A e B representam os vetores de características faciais.

$$\textbf{similaridade cosseno} = \frac{\mathbf{A} \cdot \mathbf{B}}{\| \mathbf{A} \| \| \mathbf{B} \|}$$

- **Distância Euclidiana (L2)**: a distância euclidiana (ou L2) mede a distância direta entre dois pontos no espaço multidimensional. Valores menores indicam que os vetores são mais próximos (mais semelhantes). Como indica a equação a seguir, onde os vetores de características das imagens são A e B.

$$\mathbf{L2} = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}$$

- **Verificação da Similaridade**: baseado nas pontuações de similaridade, determina-se se as imagens pertencem à mesma pessoa;
- **Desenho e Exibição**: desenha-se os rostos detectados e o resultado da comparação é exibido na imagem.

O resultado esperado deve ser semelhante às figuras a seguir, onde caso a imagem de teste não pertença à mesma pessoa, vai ficar destacado em vermelho. E se for a mesma pessoa, vai retornar o boundBox em verde.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 5 – Reconhece faces pessoas diferentes
Fonte: autor (2024)

Figura 6 – Reconhece faces da mesma pessoa
Fonte: autor (2024)

A utilização de redes pré-treinadas para reconhecimento facial permite a prototipagem e validação de forma simples e relativamente rápida. No entanto, ao estudar deep learning, é fundamental não se limitar apenas à aplicação de técnicas prontas. É essencial compreender como realizar na prática o treinamento de redes neurais para conjuntos de dados customizados. Esse é o tema que exploraremos na próxima seção.

2.3 Treinamento de rede neural para reconhecimento facial

Uma CNN é adequada para aplicações como reconhecimento de imagem e é frequentemente usada em software de reconhecimento facial. Em uma CNN, temos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

várias camadas contendo vários filtros (ou kernels, como são comumente chamados) encarregados de detectar características específicas do alvo que você está tentando detectar. A camada inicial tenta focar em recursos amplos (bordas e contornos), enquanto as últimas camadas tentam detectar recursos muito específicos (olhos, boca). Em uma CNN, os valores dos vários filtros em cada camada convolucional são obtidos treinando em um conjunto de treinamento específico. No final do treinamento, temos um conjunto de valores de filtro que serão usados para detectar recursos específicos no conjunto de dados, chamamos de modelo treinado. Usando esse modelo, podemos aplicá-lo em novas imagens para poder fazer uma previsão sobre o que está contido na imagem.

2.3.1 Explorando o modelo VGG-Face

A VGG-Face é uma rede neural convolucional (CNN) projetada para reconhecimento facial. Ela foi desenvolvida pela Visual Geometry Group (VGG), da Universidade de Oxford. A VGG-Face é amplamente utilizada em aplicações de reconhecimento facial devido à sua precisão e eficiência. Ela é baseada na famosa arquitetura VGG-16, que é uma rede neural profunda com 16 camadas treináveis. A principal diferença é que a VGG-Face foi treinada especificamente em um grande conjunto de dados de rostos, tornando-a particularmente eficaz para tarefas de reconhecimento facial. O conjunto de dados é composto principalmente por celebridades, figuras públicas, atores e políticos. Seus nomes foram extraídos da lista de celebridades do Internet Movie Data Base (IMDB), com base em gênero, popularidade, pose, iluminação, etnia e profissão (atores, atletas, políticos). As imagens desses nomes foram obtidas no Google Image Search e várias imagens para cada nome foram baixadas, examinadas por humanos e depois rotuladas para treinamento. Existem duas versões do VGGFace:

- VGG-Face: desenvolvido em 2015, treinado em 2,6 milhões de imagens, um total de 2.622 pessoas;
- VGGFace2: desenvolvido em 2017, treinado em 3,31 milhões de imagens, um total de 9.131 pessoas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Saiba mais: para conhecer sobre a vgg-face, acesse o artigo da publicação e o poster de apresentação no congresso em:

Artigo: https://www.robots.ox.ac.uk/~vgg/publications/2015/Parkhi15/parkhi15.pdf

Poster: https://www.robots.ox.ac.uk/~vgg/publications/2015/Parkhi15/poster.pdf

Vamos compreender como é a estrutura da VGG-Face. A figura a seguir representa a arquitetura que é composta por: Camadas Convolucionais (em azul), que extraem características dos rostos nas imagens de entrada, seguida por camada de Max pooling (em vermelho), que após cada bloco de camadas convolucionais, reduzem a dimensionalidade espacial das características. O final da rede possui uma função de ativação Softmax (em cinza) com o vetor de características da imagem formado.

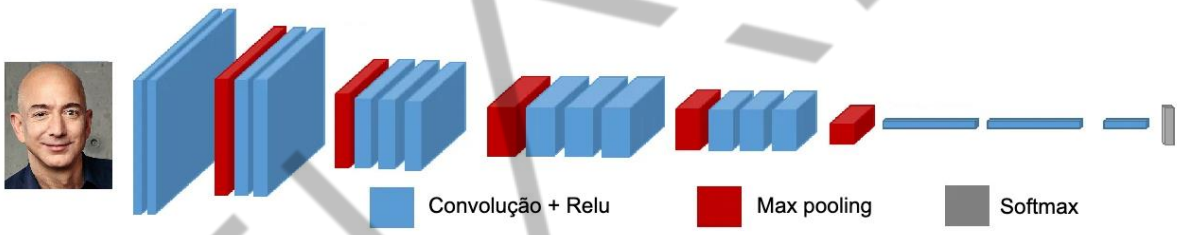

Figura 7 – arquitetura vgg-face
Fonte: autor (2024)

Agora que já conhecemos a arquitetura da VGG-face, podemos transcrever essa arquitetura para código usando o Tensorflow/keras. O Código a seguir demonstra a implementação dessa arquitetura por meio da função vgg_face.

```
      from tensorflow.keras.models import Sequential
      from tensorflow.keras.layers import ZeroPadding2D,
Convolution2D, MaxPooling2D, Dropout, Flatten, Activation

def vgg_face():
    model = Sequential()
    model.add(ZeroPadding2D((1,1),input_shape=(224,224, 3)))
    model.add(Convolution2D(64, (3, 3), activation='relu'))
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D((2,2), strides=(2,2)))
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(128, (3, 3), activation='relu'))
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(128, (3, 3), activation='relu'))
    model.add(MaxPooling2D((2,2), strides=(2,2)))

    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(256, (3, 3), activation='relu'))
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(256, (3, 3), activation='relu'))
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(256, (3, 3), activation='relu'))
    model.add(MaxPooling2D((2,2), strides=(2,2)))

    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(512, (3, 3), activation='relu'))
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(512, (3, 3), activation='relu'))
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(512, (3, 3), activation='relu'))
    model.add(MaxPooling2D((2,2), strides=(2,2)))

    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(512, (3, 3), activation='relu'))
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(512, (3, 3), activation='relu'))
    model.add(ZeroPadding2D((1,1)))
    model.add(Convolution2D(512, (3, 3), activation='relu'))
    model.add(MaxPooling2D((2,2), strides=(2,2)))

    model.add(Convolution2D(4096, (7, 7), activation='relu'))
    model.add(Dropout(0.5))
    model.add(Convolution2D(4096, (1, 1), activation='relu'))
    model.add(Dropout(0.5))
    model.add(Convolution2D(2622, (1, 1)))
    model.add(Flatten())
    model.add(Activation('softmax'))
    return model
```

Código-fonte 4 – vgg-face
Fonte: autor (2024)

Com a arquitetura VGG-Face definida, podemos seguir para a etapa de treinamento. No entanto, é importante notar que o treinamento dessa rede não será rápido nem barato; será necessário um investimento significativo em recursos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

computacionais. No próprio artigo, os autores informam que levaram cerca de 7 dias para o treinamento utilizando GPUs Nvidia Titan Black. Por essa razão, o grupo de pesquisa compartilha os pesos pré-treinados na página do grupo, acessível em https://vwww.robots.ox.ac.uk/~vgg/software/vgg_face/. No entanto, esses pesos são compatíveis com MATLAB.

Para utilizar com TensorFlow/Keras, é necessário converter os pesos para a extensão `.h5`. O modelo já convertido possui cerca de 580Mb e pode ser baixado no seguinte link: https://github.com/serengil/deepface_models/releases/download/v1.0/vgg_face_weights.h5. Vamos utilizar os pesos já convertidos para a extensão compatível com TensorFlow/Keras.

2.3.2 Explorando a Representação de Imagens com VGG-Face

Vamos compreender na prática como que é realizada a comparação entre imagens. Para isso, vamos carregar o modelo VGG-Face com os pesos pré-treinados e usaremos a penúltima camada da saída da VGG-Face para representar imagens. Isso nos permitirá obter vetores de características que podem ser usados para comparação e verificação de faces.

A VGG-Face espera imagens de entrada no formato 224x224x3, onde a terceira dimensão refere-se ao número de canais de cores (RGB). A função preprocess_input normaliza as entradas na escala de [-1, +1].

Primeiro, realizamos alguns importes de bibliotecas e definimos um novo modelo que utiliza a penúltima camada da VGG-Face:

```
      import numpy as np
import cv2
import tensorflow as tf
from keras.models import Model, Sequential
from keras.layers import ZeroPadding2D, Convolution2D, MaxPooling2D,
Flatten, Dropout, Activation
from keras.preprocessing.image import load_img, img_to_array
from keras.applications.imagenet_utils import preprocess_input
import matplotlib.pyplot as plt

def preprocess_image(image_path):
    """Preprocessa a imagem para ser usada no modelo."""
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    img = load_img(image_path, target_size=(224, 224))
    img = img_to_array(img)
    img = np.expand_dims(img, axis=0)
    img = preprocess_input(img)
    return img

      def vgg_face_descriptor(weights_path='vgg_face_weights.h5'):
    """carrega o modelo VGG-Face pré-treinado com os pesos
especificados."""
    model = Sequential([
        ZeroPadding2D((1, 1), input_shape=(224, 224, 3)),
        Convolution2D(64, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(64, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),

        ZeroPadding2D((1, 1)),
        Convolution2D(128, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(128, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),

        ZeroPadding2D((1, 1)),
        Convolution2D(256, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(256, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(256, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),

        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),

        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),

        Convolution2D(4096, (7, 7), activation='relu'),
        Dropout(0.5),
        Convolution2D(4096, (1, 1), activation='relu'),
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
        Dropout(0.5),
        Convolution2D(2622, (1, 1)),
        Flatten(),
        Activation('softmax')
    ])

    model.load_weights(weights_path)
    # A camada de ativação softmax é removida para obter a
representação facial
    vgg_face_descriptor = Model(inputs=model.input,
outputs=model.layers[-2].output)

    return vgg_face_descriptor
```

Código-fonte 5 – Carrega modelo
Fonte: autor (2024)

Note que em: vgg_face_descriptor = Model(inputs=model.input, outputs=model.layers[-2].output) é usada para criar um modelo que gera as características dos rostos em vez de classificar os rostos, o que será importante para a tarefa de reconhecimento facial. Com esse vetor de características, vamos calcular a similaridade entre diferentes representações de rostos para verificar se pertencem à mesma pessoa.

Agora que temos o modelo carregado, podemos representar imagens como um vetor de 2622 dimensões, vamos carregar as duas imagens da figura a seguir para o nosso teste, note que representam a mesma pessoa.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 8 – imagens de teste
Fonte: autor (2024)

Após o carregamento do modelo com os pesos treinados, podemos usar a função predict para obter o vetor de características das imagens.

```
      # Carrega VGG-Face model
      model = vgg_face_descriptor()

      # Carrega imagens de teste
    img1_representation=model.predict(preprocess_image('img1.png'))[0,
:]
    img2_representation=model.predict(preprocess_image('img2.png'))[0,
:]
```

Código-fonte 6 – Carrega imagens e obtém vetor de características
Fonte: autor

Para ilustrar as representações vetoriais das imagens, podemos usar gráficos de barras, vamos criar uma função auxiliar que exibe esses vetores. Isso vai ajudar a entender visualmente como os vetores de características se parecem.

```
import matplotlib.pyplot as plt

def plot_representations(rep1, rep2, title1='Imagem 1 Representação',
title2='Imagem 2 Representação'):
    fig, ax = plt.subplots(3, 1, figsize=(15, 8))
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    ax[0].bar(range(len(rep1)), rep1, color='b', alpha=0.6,
label='Image 1')
    ax[0].set_title(title1)
    ax[0].set_xlabel('Feature Index')
    ax[0].set_ylabel('Feature Value')
    ax[0].legend()

    ax[1].bar(range(len(rep2)), rep2, color='r', alpha=0.6,
label='Image 2')
    ax[1].set_title(title2)
    ax[1].set_xlabel('Feature Index')
    ax[1].set_ylabel('Feature Value')
    ax[1].legend()

    # Plot sobreposição de características das imagens 1 e 2 para
comparação visual direta
    ax[2].bar(range(len(rep1)), rep1, color='b', alpha=0.6,
label='Imagem 1')
    ax[2].bar(range(len(rep2)), rep2, color='r', alpha=0.6,
label='Imagem 2')
    ax[2].set_title('Comparação de Representações de Imagens')
    ax[2].set_xlabel('Feature Index')
    ax[2].set_ylabel('Feature Value')
    ax[2].legend()

    plt.tight_layout()
    plt.show()

# Exibir as representações das imagens
plot_representations(img1_representation, img2_representation)
```

Código-fonte 7 – exibe vetor de características
Fonte: autor (2024)

É esperado que o resultado seja um vetor de características semelhante ao da figura a seguir. Nesse gráfico, o eixo x representa o índice do vetor, enquanto o eixo y indica o valor da característica naquele ponto específico. A representação é feita com o vetor da imagem 1 em azul, o vetor da imagem 2 em vermelho e a terceira imagem mostra a sobreposição dos dois vetores para permitir uma comparação visual mais clara.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

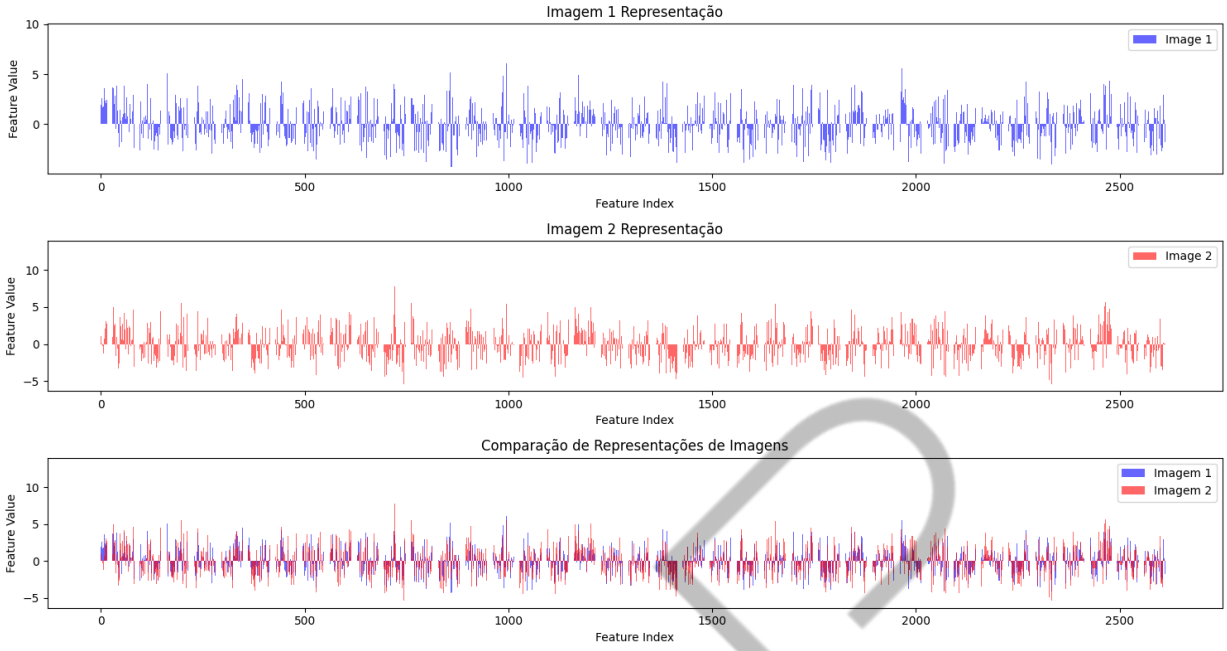

Figura 9 – gráfico de características da mesma pessoa
Fonte: autor (2024)

A título de comparação, a figura a seguir exibe o mesmo resultado para duas imagens com faces de pessoas diferentes.

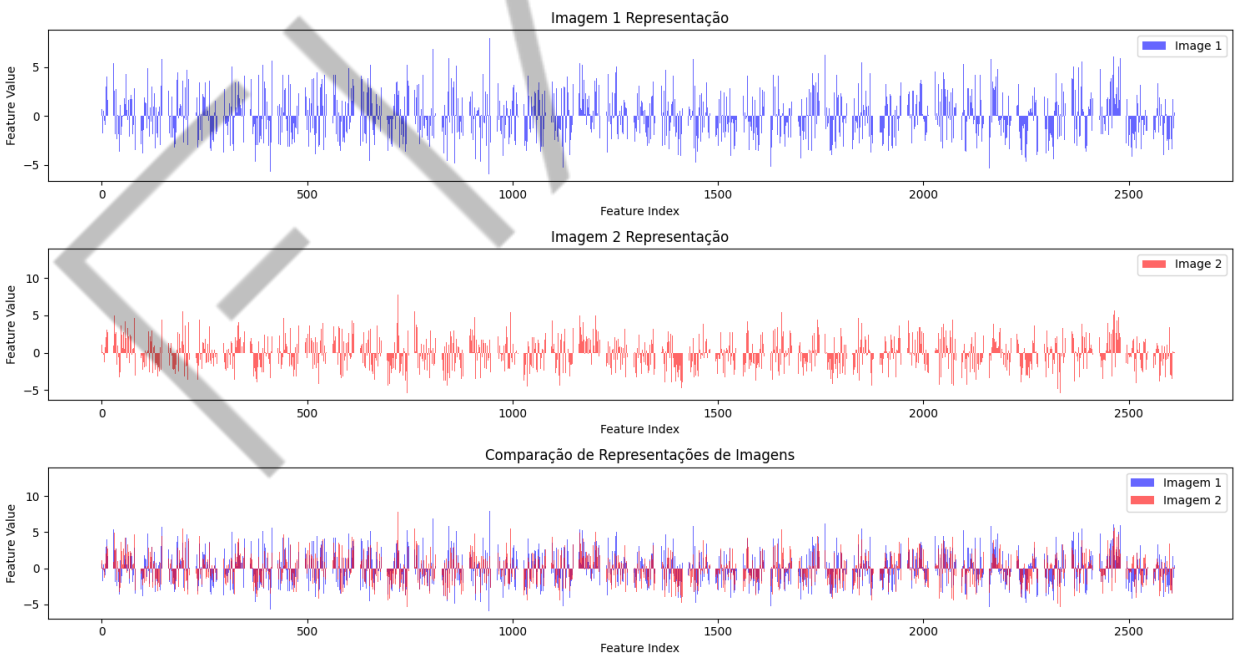

Figura 10 – gráfico de características pessoas diferentes
Fonte: autor (2024)

Nessa simples comparação visual, é possível notar que, no primeiro caso, os gráficos de barras, que representam os vetores de características, são mais semelhantes, indicando que as imagens podem se tratar da mesma pessoa. No

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

segundo caso, os gráficos são mais diferentes, sugerindo que as imagens podem ser de pessoas diferentes.

2.3.3 Explorando a Similaridade de Vetores com VGG-Face

Para ter uma resposta mais clara e objetiva, podemos calcular a similaridade entre esses vetores a fim de determinar o quão próximos eles são. Isso irá nos ajudar a decidir se duas imagens são da mesma pessoa. Existem duas maneiras comuns de medir a distância entre dois vetores: distância cosseno e distância euclidiana. A distância cosseno é igual a 1 menos a similaridade cosseno.

```
def find_cosine_distance(source_representation, test_representation):
    a = np.dot(source_representation, test_representation)
    b = np.linalg.norm(source_representation) *
np.linalg.norm(test_representation)
    return 1 - (a / b)

def find_euclidean_distance(source_representation,
test_representation):
    euclidean_distance = np.linalg.norm(source_representation -
test_representation)
    return euclidean_distance

cosine_similarity = find_cosine_distance(img1_representation,
img2_representation)
euclidean_distance = find_euclidean_distance(img1_representation,
img2_representation)

print(cosine_similarity, euclidean_distance)
```

Código-fonte 8 – calcula similaridade
Fonte: autor (2024)

A utilização das métricas de distância do cosseno ou distância euclidiana são abordagens comuns para determinar a similaridade entre vetores de características. Essas métricas permitem quantificar o quão próximos ou distantes os vetores estão entre si, auxiliando na decisão de verificação de identidade. No nosso exemplo do primeiro caso, com imagens que são da mesma pessoa, os resultados foram de 0,28

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

para distância do cosseno e 79 para distância euclidiana. Já para imagens que não são da mesma pessoa, os resultados foram 0,48 e 116, respectivamente.

A realização de testes é importante para ajudar a determinar um valor de limiar adequado que consiga determinar se as imagens são ou não da mesma pessoa. Um bom chute inicial desse valor é utilizar um valor próximo a 0,4 para distância do cosseno e 110 para distância euclidiana.

2.3.3 Reconhecimento em tempo real com VGG-Face

Agora que exploramos cada elemento do funcionamento do reconhecimento de facial com a VGG-Face, vamos criar um sistema que realiza as predições e retorna se as imagens são de funcionários de uma empresa. Esse processo é conhecido por aprendizado one-shot. Isso significa que não alimentamos várias imagens de uma pessoa na rede. Um exemplo clássico é se armazenamos uma foto de uma pessoa em nosso banco de dados e tiramos uma foto dela na entrada de um prédio para verificá-la.

Crie uma pasta chamada `faces`. Nessa pasta, vai conter uma imagem de cada funcionário; o nome da imagem deve ser o nome do funcionário, por exemplo `Jeff.jpg`. Isso é importante, pois vamos criar uma função load_employee_representations, que será responsável por varrer essa pasta e encontrar as imagens de cada funcionário.

Dica: Ao adicionar as imagens de referência na pasta faces, opte por imagens em que a pessoa esteja mais centralizada e alinhada na figura, evite imagens com qualidade ruim ou resolução baixa.

A figura a seguir exibe as imagens que serão utilizadas. Vamos fazer o reconhecimento de face de Jeff bezos e Elon Musk.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 11 – Imagens da pasta faces
Fonte: autor (2024)

A estrutura de pastas do projeto segue como apresentado pela imagem a seguir. Não se esqueça de realizar o download dos pesos da rede VGG-Face no formato H5. O arquivo de vídeo Jeff.mp4 pode ser substituído por outros vídeos com outras extensões, a OpenCV permite utilizar webcam para realizar o processamento em tempo real.

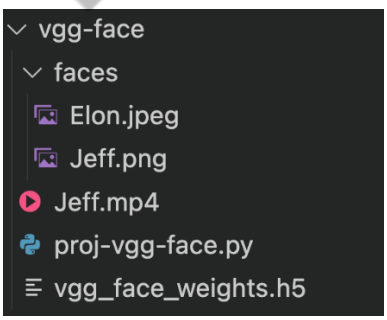

Figura 12 – Estrutura de pastas do projeto
Fonte: autor (2024)

O código principal está organizado por funções que são chamadas a cada etapa do código. Vamos ao código.

```
import numpy as np
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
import cv2
import tensorflow as tf
from keras.models import Model, Sequential
from keras.layers import ZeroPadding2D, Convolution2D, MaxPooling2D,
Flatten, Dropout, Activation
from keras.preprocessing.image import load_img, img_to_array
from keras.applications.imagenet_utils import preprocess_input
from os import listdir
import os

# Parâmetros globais
COLOR = (0, 255, 0)
EPSILON = 0.30  # Limiar de similaridade coseno
CASCADE_PATH = cv2.data.haarcascades +
"haarcascade_frontalface_default.xml"

# Inicializa o classificador cascade
face_cascade = cv2.CascadeClassifier(CASCADE_PATH)

def preprocess_image(image_path):
    """Pré-processa uma imagem para o modelo VGG-Face."""
    img = load_img(image_path, target_size=(224, 224))
    img = img_to_array(img)
    img = np.expand_dims(img, axis=0)
    return preprocess_input(img)  # Normaliza a entrada na escala de
[-1, +1]

def load_vgg_face_model(weights_path='vgg_face_weights.h5'):
    """Carrega o modelo VGG-Face com os pesos fornecidos."""
    model = Sequential([
        ZeroPadding2D((1, 1), input_shape=(224, 224, 3)),
        Convolution2D(64, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(64, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),
        ZeroPadding2D((1, 1)),
        Convolution2D(128, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(128, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),
        ZeroPadding2D((1, 1)),
        Convolution2D(256, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(256, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(256, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        ZeroPadding2D((1, 1)),
        Convolution2D(512, (3, 3), activation='relu'),
        MaxPooling2D((2, 2), strides=(2, 2)),
        Convolution2D(4096, (7, 7), activation='relu'),
        Dropout(0.5),
        Convolution2D(4096, (1, 1), activation='relu'),
        Dropout(0.5),
        Convolution2D(2622, (1, 1)),
        Flatten(),
        Activation('softmax')
    ])

    model.load_weights(weights_path)
    return Model(inputs=model.input, outputs=model.layers[-2].output)

def find_cosine_similarity(source_representation,
test_representation):
    """Calcula a similaridade coseno entre duas representações."""
    a = np.dot(source_representation, test_representation)
    b = np.linalg.norm(source_representation) *
np.linalg.norm(test_representation)
    return 1 - (a / b)

def load_employee_representations(model, employee_pictures_path):
    """Carrega as representações dos funcionários."""
    employees = {}
    for file in listdir(employee_pictures_path):
        try:
            if file.lower().endswith(('.png', '.jpg', '.jpeg')):
                employee, _ = file.split(".")
                employees[employee] =
model.predict(preprocess_image(os.path.join(employee_pictures_path,
file)))[0, :]
        except Exception as e:
            print(f"Error processing file {file}: {e}")
    return employees

def recognize_faces_in_video(video_path, model, employees):
    """Reconhece faces em um vídeo."""
    cap = cv2.VideoCapture(video_path)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    while cap.isOpened():
        ret, img = cap.read()
        if not ret:
            break

        img = cv2.resize(img, (int(0.5 * img.shape[1]), int(0.5 *
img.shape[0])), interpolation=cv2.INTER_AREA)
        faces = face_cascade.detectMultiScale(img, 1.2, 5)

        for (x, y, w, h) in faces:
            if w > 20:
                cv2.rectangle(img, (x, y), (x + w, y + h), COLOR, 4)

                detected_face = img[y:y+h, x:x+w]
                detected_face = cv2.resize(detected_face, (224, 224))
                img_pixels = img_to_array(detected_face)
                img_pixels = np.expand_dims(img_pixels, axis=0)
                img_pixels = preprocess_input(img_pixels)

                captured_representation = model.predict(img_pixels)[0,
:]

                found = False
                for employee_name, representation in
employees.items():
                    similarity =
find_cosine_similarity(representation, captured_representation)
                    if similarity < EPSILON:
                        label_name = f"{employee_name}
({similarity:.2f})"
                        cv2.putText(img, label_name, (x + w, y ),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, COLOR, 2)
                        found = True
                        break

        cv2.imshow('img', img)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    model = load_vgg_face_model()
    employee_pictures_path = "faces"
    employees = load_employee_representations(model,
employee_pictures_path)
    print("Representações de funcionários carregadas com sucesso")
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
video_path = 'Jeff.mp4' # para vídeo, 0 para webcam

recognize_faces_in_video(video_path, model, employees)
```

Código-fonte 9 – Reconhecimento em vídeo
Fonte: autor (2024)

O código começa carregando o modelo VGG-Face e as representações faciais dos funcionários. Em seguida, ele captura frames do vídeo. O classificador Haar Cascade detecta faces é o responsável por detectar faces de forma rápida nos frames. Quando é detectada uma face, imediatamente ela é redimensionada e normalizada, para que o modelo VGG-Face extraia a representação facial dela. A representação da face detectada é comparada com as representações dos funcionários usando similaridade cosseno. Se uma face for reconhecida, o nome do funcionário é anotado no frame e é exibido em tempo real.

Como resultado, é esperado uma saída de vídeo semelhante à figura a seguir, que demonstra o reconhecimento facial.

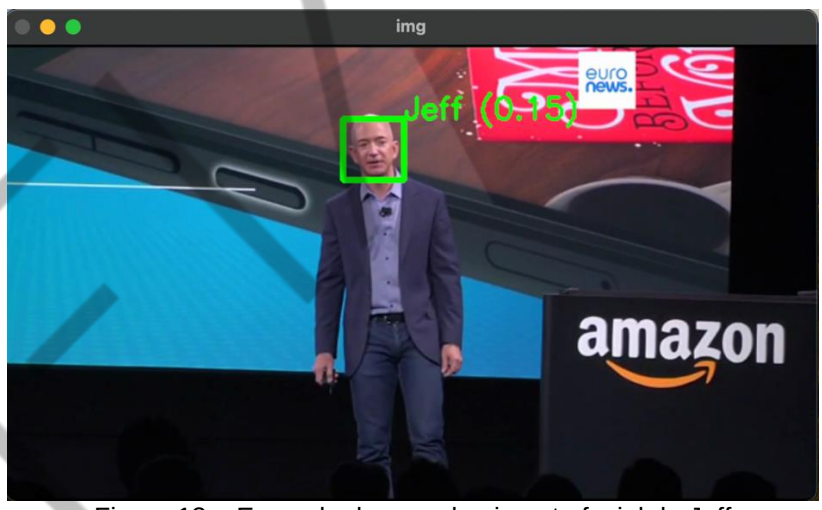

Figura 13 – Exemplo de reconhecimento facial de Jeff
Fonte: autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

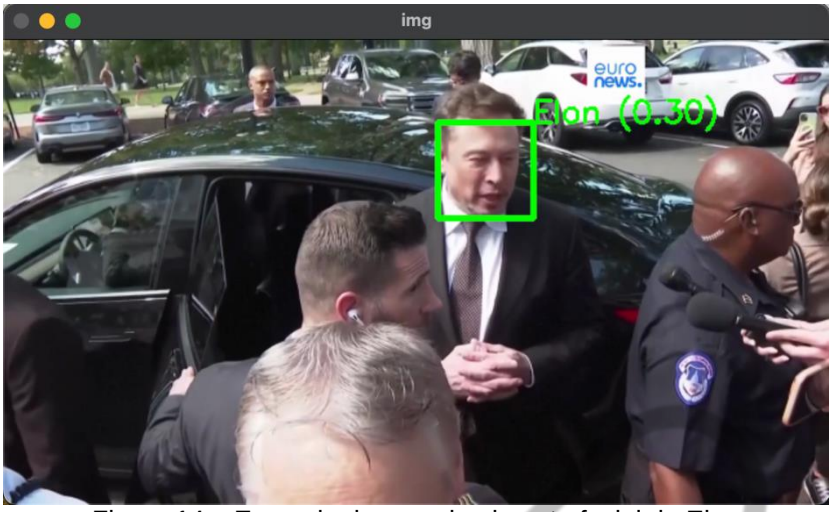

Figura 14 – Exemplo de reconhecimento facial de Elon
Fonte: autor (2024)

Pronto! Finalizamos nosso sistema de reconhecimento facial. Agora é a sua vez: realize ensaios com imagens e vídeos diferentes e compare os resultados. E que tal aprimorar um pouco mais nosso sistema de reconhecimento, tentando reconhecer as expressões faciais das pessoas? É o que vamos decifrar a seguir.

2.4 Sistema de expressões faciais

O Reconhecimento de Emoções Faciais (FER - Facial Emotion Recognition) é uma aplicação fascinante da visão computacional, essencial para diversas áreas, como interação humano-computador, análise de comportamento do consumidor, sistemas de segurança, entre outras. Trata-se de uma área de crescente interesse, envolvendo a análise de expressões faciais para identificar e categorizar sentimentos como felicidade, tristeza, surpresa, raiva, medo, nojo e neutralidade, como exibido na figura a seguir.

Figura 15 – Exemplo de expressões faciais
Fonte: autor (2024)

Vamos explorar os conceitos, técnicas e ferramentas utilizadas para criar um sistema de detecção de emoções baseado em expressões faciais.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.4.1 Carregando e pré-processando o Conjunto de Dados

Em qualquer projeto de deep learning, os dados utilizados são fundamentais para definir a precisão do treinamento e a qualidade do modelo final. Nesse sentido, é crucial ter um conjunto de dados robusto e diversificado. Alguns dos datasets mais utilizados para essa tarefa incluem:

- FER-2013: um dos datasets mais populares, contém milhares de imagens de rostos anotadas com emoções como felicidade, tristeza, surpresa, raiva, medo, nojo e neutralidade;
- CK+ (The Extended Cohn-Kanade Dataset): amplamente usado para estudos de expressão facial, contém sequências de vídeos com anotações de emoções;
- RAF-DB (Real-world Affective Faces Database): um dataset diversificado com imagens de expressões faciais coletadas em condições do mundo real;
- AffectNet: um dos maiores datasets disponíveis publicamente, contendo mais de um milhão de imagens de rostos anotadas com expressões faciais, além de anotações de Valência e Arousal, que indicam a positividade e intensidade da emoção, respectivamente.

Vamos utilizar o FER-2013, que é um dataset que contém 35.887 imagens em tons de cinza de rostos, cada uma com 48x48 pixels. As imagens são anotadas com uma das sete emoções: felicidade, tristeza, surpresa, raiva, medo, nojo e neutralidade. Esse conjunto de dados foi introduzido durante a competição ICML 2013.

Para fazer o download, acesse o link: https://www.kaggle.com/competitions/challenges-in-representation-learning-facial-expression-recognition-challenge/data, faça o cadastro na plataforma e realize o download.

Após o download, descompacte o arquivo para acessar os dados. O arquivo zip conterá um arquivo CSV (fer2013.csv) que inclui as imagens e suas respectivas anotações de emoção. A estrutura do CSV é a seguinte:

- emotion: a etiqueta da emoção (0 para raiva, 1 para nojo, 2 para medo, 3 para felicidade, 4 para tristeza, 5 para surpresa, 6 para neutralidade);

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- pixels: uma string contendo os valores dos pixels (48x48) da imagem em tons de cinza;
- Usage: indica se a imagem é para treinamento, validação ou teste.

Dica: Vamos utilizar o Google Colab para o treinamento da rede, pois temos à disposição uma GPU gratuita. Por essa razão, tenha o dataset no Google Drive para facilitar o acesso.

É necessário ter atenção para carregar corretamente os dados; ajuste o caminho para seu drive.

```
from google.colab import drive
import os
import tarfile
import pandas as pd
import numpy as np
from tensorflow.keras.utils import to_categorical

# Montar o Google Drive
drive.mount('/content/drive')

# Arquivo raiz zipado no Google Drive ### ajuste esse caminho###
! unzip -uq '/content/drive/MyDrive/fer/challenges-in-representation-
learning-facial-expression-recognition-challenge.zip' -d "/content"

# Caminho para o arquivo tar.gz
tar_path = '/content/fer2013.tar.gz'

# Caminho para extrair os arquivos
extract_path = '/content/fer2013'

# Criar diretório para extração
os.makedirs(extract_path, exist_ok=True)

# Descompactar o arquivo tar.gz
with tarfile.open(tar_path, 'r:gz') as tar:
    tar.extractall(path=extract_path)

# Verificar os arquivos extraídos
print("Arquivos extraídos:", os.listdir(extract_path))

# Caminho para o arquivo CSV extraído
csv_path = os.path.join(extract_path, 'fer2013/fer2013.csv')
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Carregar os dados
data = pd.read_csv(csv_path)
```

Código-fonte 10 – Importa dataset do google drive
Fonte: autor (2024)

Agora que temos os dados importados corretamente no google Colab, podemos seguir fazendo o carregamento dos dados extraídos e pré-processar convertendo os pixels de string para arrays de numpy e normalizando os valores. Por fim, vamos separar os dados em conjuntos de treinamento, validação e teste.

```
# Função de pré-processamento
def preprocess_data(data):
    pixels = data['pixels'].tolist()
    faces = [np.fromstring(pixel_sequence, dtype=int, sep='
').reshape(48, 48, 1) for pixel_sequence in pixels]
    faces = np.array(faces).astype('float32')
    faces /= 255.0
    emotions = to_categorical(data['emotion'], num_classes=7)
    return faces, emotions

# Separar os dados de treinamento, validação e teste
train_data = data[data['Usage'] == 'Training']
val_data = data[data['Usage'] == 'PublicTest']
test_data = data[data['Usage'] == 'PrivateTest']

x_train, y_train = preprocess_data(train_data)
x_val, y_val = preprocess_data(val_data)
x_test, y_test = preprocess_data(test_data)

# Verificar as formas dos dados
print('x_train shape:', x_train.shape)
print('y_train shape:', y_train.shape)
print('x_val shape:', x_val.shape)
print('y_val shape:', y_val.shape)
print('x_test shape:', x_test.shape)
print('y_test shape:', y_test.shape)
```

Código-fonte 11 – Carrega os dados de treinamento, validação e teste
Fonte: autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Vamos visualizar algumas imagens para nos certificar de que está tudo ocorrendo de forma correta.

```
import matplotlib.pyplot as plt
import numpy as np

# Dicionário para mapear índices de emoção para rótulos textuais
label_to_text = {0: 'angry', 1: 'disgust', 2: 'fear', 3: 'happiness',
4: 'sad', 5: 'surprise', 6: 'neutral'}

# Função para exibir algumas imagens do dataset
def show_sample_images(faces, emotions, num_images=10):
    plt.figure(figsize=(15, 5))
    for i in range(num_images):
        ax = plt.subplot(2, num_images // 2, i + 1)
        plt.imshow(faces[i].reshape(48, 48), cmap='gray')
        plt.title(label_to_text[np.argmax(emotions[i])])
        plt.axis('off')
    plt.show()

# Exibir 10 imagens de exemplo do conjunto de treinamento
show_sample_images(x_train, y_train, num_images=10)
```

Código-fonte 12 – Carrega os dados de treinamento, validação e teste
Fonte: autor

O resultado esperado deve ser semelhante à figura a seguir com as imagens do dataset.

Figura 16 – Exemplo de imagens do dataset
Fonte: autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Vamos utilizar a técnica de data augmentation que são aplicadas durante o treinamento para gerar novas variações das imagens de entrada, ajudando a evitar overfitting e tornando o modelo mais robusto a diferentes variações e distorções nas imagens reais. Elas permitem que o modelo generalize melhor para novos dados que podem não ser exatamente iguais aos dados de treinamento.

As transformações especificadas no ImageDataGenerator são:

- Rescale: normaliza os valores dos pixels para o intervalo [0, 1] dividindo-os por 255;
- rotation_range: rotaciona aleatoriamente as imagens dentro de um intervalo de 30 graus;
- width_shift_range: translada aleatoriamente as imagens horizontalmente (para a esquerda ou direita) em até 20% da largura da imagem;
- height_shift_range: translada aleatoriamente as imagens verticalmente (para cima ou para baixo) em até 20% da altura da imagem;
- shear_range: aplica uma transformação de cisalhamento aleatória às imagens;
- zoom_range: aplica um zoom aleatório nas imagens, variando até 20%;
- horizontal_flip: inverte aleatoriamente as imagens horizontalmente;
- fill_mode: estratégia para preencher os pixels gerados ou perdidos após a aplicação das transformações (como rotações ou translações). O modo 'nearest' preenche os novos pixels com o valor do pixel mais próximo.

```
from keras.preprocessing.image import ImageDataGenerator
# Criar geradores de dados
train_datagen = ImageDataGenerator(
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
val_datagen = ImageDataGenerator()
test_datagen = ImageDataGenerator()

# Criar geradores
train_generator = train_datagen.flow(x_train, y_train, batch_size=64)
validation_generator = val_datagen.flow(x_val, y_val, batch_size=64)
test_generator = test_datagen.flow(x_test, y_test, batch_size=64)
```

Código-fonte 13 – aumento de dados
Fonte: autor (2024)

Ótimo! Agora que temos os dados corretamente pré-processados, podemos dar continuidade ao treinamento da nossa rede neural.

2.4.2 Treinamento da rede neural de detecção de emoções

Utilizando o Keras/TensorFlow, vamos definir a arquitetura da nossa rede neural. A nossa rede é sequencial, composta por camadas de convolução e pooling com um classificador MLP:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout

model = Sequential([

    Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(48,
48, 1)),
    Conv2D(64, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),

    Conv2D(128, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(128, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),

    Flatten(),
    Dense(1024, activation='relu'),
    Dropout(0.5),
    Dense(7, activation='softmax')
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

Código-fonte 14 – Arquitetura da rede CNN
Fonte: autor (2024)

Vamos adicionar os callbacks de treinamento, como ModelCheckpoint, EarlyStopping, e ReduceLROnPlateau, que são úteis para salvar o melhor modelo, parar o treinamento cedo se o modelo não estiver melhorando e reduzir a taxa de aprendizado se a validação não estiver melhorando. E realizar o treinamento do nosso modelo utilizando o google Colab com a configuração da GPU ativa.

```
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping,
ReduceLROnPlateau


# Callbacks
checkpoint = ModelCheckpoint('best_model.h5', monitor='val_loss',
save_best_only=True, mode='min')
early_stopping = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2,
patience=5, min_lr=0.001)

# Treinamento do modelo usando geradores
history = model.fit(
    train_generator,
    steps_per_epoch=len(x_train) // 64,
    epochs=50,
    validation_data=validation_generator,
    validation_steps=len(x_val) // 64,
    callbacks=[checkpoint, early_stopping, reduce_lr]
)
```

Código-fonte 15 – Treinamento do modelo
Fonte: autor (2024)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Após o treinamento, podemos visualizar o resultado por meio das curvas de treinamento do modelo de Loss e Acuracy. É por meio dessas curvas que conseguimos ter uma intuição se o modelo teve um bom treinamento ou se é necessário realizar ajustes na arquitetura da rede ou nos parâmetros de aprendizagem.

```
import matplotlib.pyplot as plt

# Plot the training and validation loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend()
plt.show()

# Plot the training and validation accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend()
plt.show()
```

Código-fonte 16 – Exibe curvas de aprendizagem do modelo
Fonte: autor (2024)

Figura 17 – Resultado de treinamento do modelo
Fonte: autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Note que, com base nos gráficos de perda (loss) e precisão (accuracy), podemos observar que o modelo está se comportando de maneira esperada durante o treinamento. A curva de perda no conjunto de treinamento diminui continuamente, indicando que o modelo está aprendendo e melhorando suas previsões. A curva de perda de validação também diminui inicialmente, mas eventualmente se estabiliza. A curva de precisão no conjunto de treinamento mostra um aumento gradual, enquanto a precisão de validação aumenta rapidamente no início e depois se estabiliza, com pequenas flutuações. No geral, o modelo ficou com um acurácia de 60% (não é um resultado bom). Além disso, a acurácia é um resumo de todas as classes. Para ter uma visão melhor dos resultados, é importante visualizar a matriz de confusão que irá exibir os resultados preditos versus os reais.

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns


# Avaliação do modelo usando geradores
test_loss, test_acc = model.evaluate(x_test,y_test, steps=len(x_test)
// 64)
print('Test accuracy:', test_acc)

# Previsões
y_pred = model.predict(x_test, steps=len(x_test) // 64)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

# Calcular a matriz de confusão
conf_matrix = confusion_matrix(y_true, y_pred_classes)

# Função para plotar a matriz de confusão
def plot_confusion_matrix(cm, classes, title='Confusion Matrix',
cmap=plt.cm.Blues):
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap=cmap,
xticklabels=classes, yticklabels=classes)
    plt.title(title)
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.show()

# Plotar a matriz de confusão
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
plot_confusion_matrix(conf_matrix, classes=[label_to_text[i] for i in
range(7)])
```

Código-fonte 17 – Exibe matriz de confusão
Fonte: autor (2024)

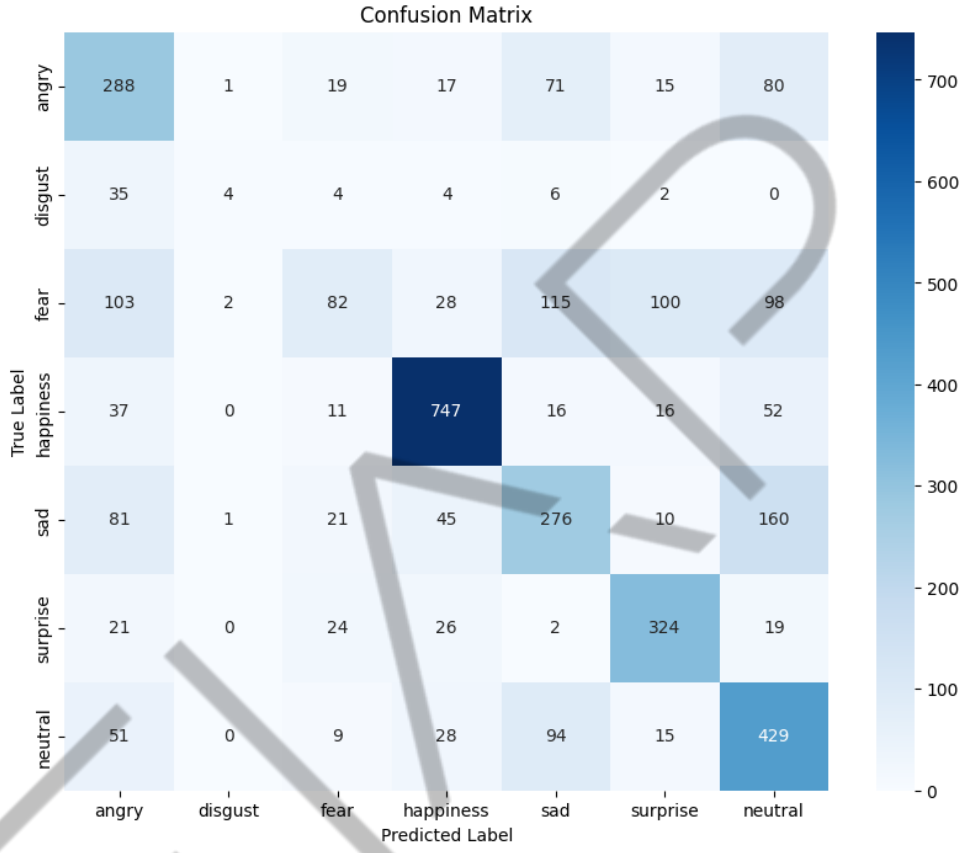

Figura 18 – Resultado de treinamento do modelo
Fonte: autor (2024)

Note que o modelo faz boas predições para as emoções de felicidade (happiness), neutra, surpresa e raiva. Para as outras emoções, o modelo tem mais dificuldades e erra muito.

Ao longo dessa seção, aprendemos e discutimos técnicas e estratégias que podemos aplicar para melhorar a qualidade do modelo; é importante que você tente otimizar a qualidade do modelo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.4.3 Detecção de emoções em tempo real

Após a otimização do treinamento do modelo, podemos usá-lo na prática. Faça o download do modelo salvo com extensão .h5 e, em seu computador local, crie um script python que carrega esse modelo e realiza as predições com os frames da webcam ou um vídeo .mp4.

```
import cv2
import numpy as np
from tensorflow.keras.models import load_model

# Dicionário para mapear índices de emoção para rótulos textuais
label_to_text = {0: 'angry', 1: 'disgust', 2: 'fear', 3: 'happiness',
4: 'sad', 5: 'surprise', 6: 'neutral'}

# Carregar o modelo treinado
model_path = 'best_model_FER.h5'
model = load_model(model_path)

# Inicializar a webcam
cap = cv2.VideoCapture(0)

def preprocess_frame(frame):
    # Convert to grayscale
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # Resize to 48x48 pixels (the input size expected by the model)
    resized_frame = cv2.resize(gray_frame, (48, 48))
    # Normalize the pixel values
    normalized_frame = resized_frame / 255.0
    # Reshape to match the input shape of the model: (1, 48, 48, 1)
    reshaped_frame = np.reshape(normalized_frame, (1, 48, 48, 1))
    return reshaped_frame

while True:
    # Capturar frame-by-frame
    ret, frame = cap.read()
    if not ret:
        break

    # Pré-processar o frame
    preprocessed_frame = preprocess_frame(frame)

    # Fazer a previsão
    predictions = model.predict(preprocessed_frame)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    emotion_index = np.argmax(predictions)
    emotion_label = label_to_text[emotion_index]

    # Desenhar a emoção prevista no frame
    cv2.putText(frame, emotion_label, (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)

    # Mostrar o frame
    cv2.imshow('Emotion Detection', frame)

    # Sair do loop ao pressionar 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Quando tudo estiver feito, libere a captura
cap.release()
cv2.destroyAllWindows()
```

Código-fonte 18 – Emoção em tempo real
Fonte: autor (2024)

Como resultado, é esperado que o modelo consiga realizar a detecção de emoções em tempo real, como exibe a figura a seguir.

Figura 19 – Resultado detecção de emoções
Fonte: autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

É possível aprimorar a precisão das detecções de emoções por meio de diversas estratégias. Um exemplo é o aumento do número de épocas de treinamento, permitindo ao modelo aprender mais profundamente as características das emoções. Outra técnica eficaz é o uso de aprendizado por transferência, onde se aplica a arquitetura de uma rede pré-treinada com os pesos já otimizados, adaptando-a para a tarefa específica de detecção de emoções. Para melhorar a qualidade das predições em tempo real, pode-se utilizar uma rede de detecção de face, como Yunet ou Haar Cascade, antes de aplicar o modelo de detecção de emoções.

Vamos avançar um pouco mais e adicionar mais inteligência ao nosso sistema. Que tal aprimorar o sistema determinando a idade e o sexo das faces reconhecidas? Vamos descobrir como.

2.5 Sistema de predição de idade e sexo

A predição de idade e sexo e outros atributos como etnia a partir de reconhecimento facial vai envolver a utilização de deep learning para analisar características faciais que indicam a faixa etária e o gênero da pessoa. Assim como sistemas de reconhecimento de emoções, esses sistemas têm aplicações em marketing, segurança, análise de comportamento, personalização de serviços, dentre outras diversas áreas.

Para desenvolver um sistema de predição de idade e sexo, é necessário um conjunto de dados diversificado e bem anotado. Os conjuntos de dados mais conhecidos são os IMDB-WIKI, Adience, e UTKFace. Vamos detalhar cada um deles:

- IMDB-WIKI:
    - Descrição: o dataset IMDB-WIKI é um dos maiores datasets públicos disponíveis para predição de idade e gênero. Ele contém mais de 500.000 imagens de celebridades coletadas do IMDb e da Wikipedia;
    - Anotações: cada imagem está anotada com a idade (calculada com base no ano de nascimento fornecido) e o gênero;
    - Formato: as imagens variam em resolução e qualidade, refletindo a diversidade das fontes de onde foram obtidas;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

    - Aplicações: esse dataset é amplamente utilizado para treinar modelos de aprendizado profundo para predição de idade e gênero devido à sua grande quantidade de dados e variedade.

- Adience:
    - Descrição: o dataset Adience foi criado para avaliar algoritmos de classificação etária e de gênero em condições reais. Ele contém imagens coletadas de álbuns de fotos não profissionais;
    - Anotações: as imagens são anotadas com faixas etárias (por exemplo, 0-2, 4-6, 8-13, 15-20, 25-32, 38-43, 48-53, 60+ anos) e gênero;
    - Formato: as imagens são capturadas em ambientes não controlados, com variações de iluminação, expressões faciais, poses e oclusões;
    - Aplicações: ideal para avaliar a robustez dos algoritmos de predição de idade e gênero em cenários do mundo real, onde as condições não são ideais.
- UTKFace:
    - Descrição: o dataset UTKFace contém quase 24 mil imagens de rostos com anotações detalhadas de idade, gênero e etnia;
    - Anotações: as imagens são anotadas com a idade exata, gênero (masculino ou feminino) e etnia (branco, negro, asiático, indiano, outros);
    - Formato: as imagens têm resolução variada, e a diversidade do dataset é refletida nas diferentes idades, gêneros e etnias representadas;
    - Aplicações: esse dataset é útil para treinar e avaliar modelos que precisam lidar com diversidade étnica, além de predição de idade e gênero.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.5.1 Conhecendo e preparando os Dados

Vamos utilizar o dataset UTKFace, que possui três atributos de predição, idade, gênero e etnia. O download do dataset pode ser feito no link https://susanqq.github.io/. O dataset está organizado com todas as imagens dentro do mesmo diretório. Todas as informações estão contidas no nome das imagens. Por exemplo, a imagem **UTKFace/9_1_4_20170103213057382.jpg.chip.jpg** significa que:

- **UTKFace/** é um prefixo;
- **9** é a idade;
- **1** é o gênero (0: masculino, 1: feminino);
- **4** é a raça (0: Branco, 1: Negro, 2: Asiático, 3: Indiano, 4: Outro);
- O restante é a data e a extensão da imagem (jpg).

De forma geral as imagens são: **[idade] _ [gênero] _ [raça] _ [data&hora].jpg**

Agora que já entendemos os parâmetros dos dados, podemos começar nosso projeto com o google colab. Primeiramente, vamos fazer o download do dataset e exibir algumas imagens.

```
# Faz o download do dataset UTKFace
!wget https://hydranets-data.s3.eu-west-3.amazonaws.com/UTKFace.zip
!jar xf UTKFace.zip

# Carrega algumas bibliotecas
import os
import glob
import random
from PIL import Image
import matplotlib.pyplot as plt

# Obter a lista de caminhos de imagem
image_paths = sorted(glob.glob("UTKFace/*.jpg.chip.jpg"))

print(f"Primeira imagem: {image_paths[0]}")
print(f"Última imagem: {image_paths[-1]}")
print(f"Total de imagens: {len(image_paths)}")

# Selecionar algumas imagens aleatórias
num_images_to_show = 5
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
random_image_paths = random.sample(image_paths, num_images_to_show)

# Visualizar imagens aleatórias
fig, axes = plt.subplots(1, num_images_to_show, figsize=(20, 5))
for i, img_path in enumerate(random_image_paths):
    img = Image.open(img_path)
    axes[i].imshow(img)
    axes[i].axis('off')
    axes[i].set_title(os.path.basename(img_path))
plt.tight_layout()
plt.show()
```

Código-fonte 19 – Carrega dataset UTKFace
Fonte: autor (2024)

O resultado esperado é a visualização de algumas imagens aleatórias do conjunto de dados para começarmos a nos familiarizar, como exibe a figura a seguir.

Figura 20 – exemplo imagens do dataset UTKFace
Fonte: autor (2024)

Note que, diferentemente dos outros conjuntos de dados, o UTKFace não está organizado por pastas que separam os rótulos. Nesse caso, o nome da imagem que possui as informações dos atributos. Dessa forma, é importante que façamos algumas etapas para correta criação do dataset para o treinamento do modelo. Além de realizar a divisão dos dados em treinamento e teste, temos que criar os rótulos de idade, gênero e raça para o treinamento.

```
# Labels - age, gender, ethnicity
image_paths = []
age_labels = []
gender_labels = []
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
ethnicity_labels = []

# varre todas as imagens criando os rotulos
for filename in os.listdir(BASE_DIR):
    image_path = os.path.join(BASE_DIR, filename)
    temp = filename.split('_')
    if len(temp) == 4:  # para garantir que split está correto
        try:
            age = int(temp[0])
            gender = int(temp[1])
            ethnicity = int(temp[2].split('.')[0])
            image_paths.append(image_path)
            age_labels.append(age)
            gender_labels.append(gender)
            ethnicity_labels.append(ethnicity)
        except ValueError:
            continue  # Skip se estiver nomeado errado

# Converte para DataFrame
df = pd.DataFrame({
    'image': image_paths,
    'age': age_labels,
    'gender': gender_labels,
    'ethnicity': ethnicity_labels
})
df.head()

from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(df, test_size=0.3,
random_state=42)

print(f"Tamanho dos dados de treino: {len(train_df)}")
print(f"Tamanho dos dados de teste: {len(test_df)}")
```

Código-fonte 20 – ajusta o dataset
Fonte: autor (2024)

O resultado esperado deve ser um dataframe com os atributos em colunas e divididos em dados de treino e teste.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	image	age	gender	ethnicity
0	UTKFace/10_1_0_20170109204259563.jpg.chip.jpg	10	1	0
1	UTKFace/35_1_0_20170117174927240.jpg.chip.jpg	35	1	0
2	UTKFace/47_0_3_20170117190238883.jpg.chip.jpg	47	0	3
3	UTKFace/49_1_0_20170104184001972.jpg.chip.jpg	49	1	0
4	UTKFace/40_0_0_20170117182249476.jpg.chip.jpg	40	0	0

Figura 21 – exemplo do dataframe ajustado
Fonte: autor (2024)

2.5.2 Definindo a arquitetura do modelo

Com dados pré-processados, estamos prontos para criar a arquitetura do nosso modelo. A nossa arquitetura será do tipo múltiplas tarefas (Multitask Learning - MTL), que é uma abordagem no campo do aprendizado de máquina, onde um único modelo é treinado para realizar várias tarefas simultaneamente. Em vez de treinar modelos separados para cada tarefa, por exemplo, uma rede para predizer idade, outra para etnia e uma terceira para raça, o Multitask Learning explora a inter-relação entre tarefas para melhorar o desempenho geral. A ideia central é que, ao aprender várias tarefas ao mesmo tempo, o modelo pode compartilhar representações úteis e padrões comuns entre essas tarefas, levando a uma generalização melhorada e uma maior eficiência. De forma geral, o compartilhamento de informações entre tarefas pode ajudar o modelo a generalizar melhor, especialmente em situações em que há poucas amostras para algumas das tarefas.

Estrutura de Redes Multitarefa:

Uma rede multitask possui uma base compartilhada seguida por ramificações específicas para cada tarefa. Cada ramificação contém camadas e parâmetros dedicados à tarefa correspondente. A base compartilhada ajuda a aprender características comuns, enquanto as ramificações específicas refinam essas características para a tarefa particular, como mostra a figura a seguir.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

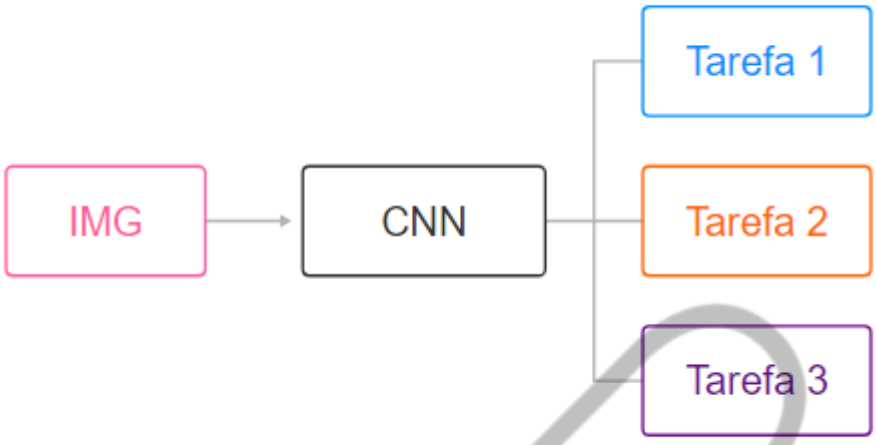

Figura 22 – exemplo de rede multitask learning
Fonte: autor (2024)

Com a API do TensorFlow, é possível criar pipelines de dados eficientes e escaláveis. O tf.data.Dataset permite que os dados sejam carregados, pré-processados e alimentados ao modelo de maneira eficiente, utilizando técnicas como paralelismo e prefetching para maximizar a utilização dos recursos de hardware.

Vamos utilizar esses recursos para a criação e o treinamento do nosso modelo.

```
# Model Definition
# Define the input
input_img = Input(shape=(128, 128, 3))  # Reduced image size

# Define a base da rede neural
x = Conv2D(32, (3, 3), activation='relu')(input_img)
x = MaxPooling2D((2, 2))(x)
x = Flatten()(x)

# Vamos definer as saidas da rede
age_output = Dense(1, name='age')(x)
gender_output = Dense(1, activation='sigmoid', name='gender')(x)
ethnicity_output = Dense(5, activation='softmax', name='ethnicity')(x)

# Criamos o nosso modelo
model = Model(inputs=input_img, outputs=[age_output, gender_output,
ethnicity_output])
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Compilamos o modelo, cada saída tem sua própria configuração
model.compile(
    optimizer='adam',
    loss={'age': 'mean_squared_error', 'gender':
'binary_crossentropy', 'ethnicity':
'sparse_categorical_crossentropy'},
    metrics={'age': 'mae', 'gender': 'accuracy', 'ethnicity':
'accuracy'}
)

# função para pré-processar as imagens
def load_and_preprocess_image(file_path):
    image = tf.io.read_file(file_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, [128, 128])  # Reduced image size
    image = image / 255.0
    return image

def load_and_preprocess_from_path_label(data):
    return load_and_preprocess_image(data['image']), (data['age'],
data['gender'], data['ethnicity'])

# Cria o dataset a partir do Dataframe
batch_size = 8  # tamanho do batch

train_data = tf.data.Dataset.from_tensor_slices(dict(train_df))
train_data = train_data.map(load_and_preprocess_from_path_label,
num_parallel_calls=tf.data.experimental.AUTOTUNE)
train_data =
train_data.cache().shuffle(buffer_size=1000).batch(batch_size).prefetc
h(buffer_size=tf.data.experimental.AUTOTUNE)

test_data = tf.data.Dataset.from_tensor_slices(dict(test_df))
test_data = test_data.map(load_and_preprocess_from_path_label,
num_parallel_calls=tf.data.experimental.AUTOTUNE)
test_data =
test_data.cache().batch(batch_size).prefetch(buffer_size=tf.data.exper
imental.AUTOTUNE)

# Treina o modelo
history = model.fit(
    train_data,
    epochs=10,
    validation_data=test_data
)

# Salva o modelo
model.save('modelo.h5')
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Código-fonte 21 – Criação da rede e treinamento do modelo
Fonte: autor (2024)

Com o modelo treinado, podemos visualizar a Loss e a acurácia do modelo ao longo do treinamento.

```
import matplotlib.pyplot as plt

def plot_training_curves(history):
    metrics = {
        'Loss Total': ['loss', 'val_loss'],
        'Loss de Idade': ['age_loss', 'val_age_loss'],
        'Loss de Gênero': ['gender_loss', 'val_gender_loss'],
        'Loss de Etnia': ['ethnicity_loss', 'val_ethnicity_loss'],
        'Acurácia de Gênero': ['gender_accuracy',
'val_gender_accuracy'],
        'Acurácia de Etnia': ['ethnicity_accuracy',
'val_ethnicity_accuracy']
    }

    plt.figure(figsize=(12, 10))

    for i, (title, keys) in enumerate(metrics.items()):
        plt.subplot(3, 2, i+1)
        plt.plot(history.history[keys[0]], label=f'Treino {title}')
        plt.plot(history.history[keys[1]], label=f'Validação {title}')
        plt.xlabel('Épocas')
        plt.ylabel(title)
        plt.legend()
        plt.title(title)

    plt.tight_layout()
    plt.show()

# curvas de treinamento
plot_training_curves(history)
```

Código-fonte 22 – Função para visualizar as curvas de treinamento
Fonte: autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

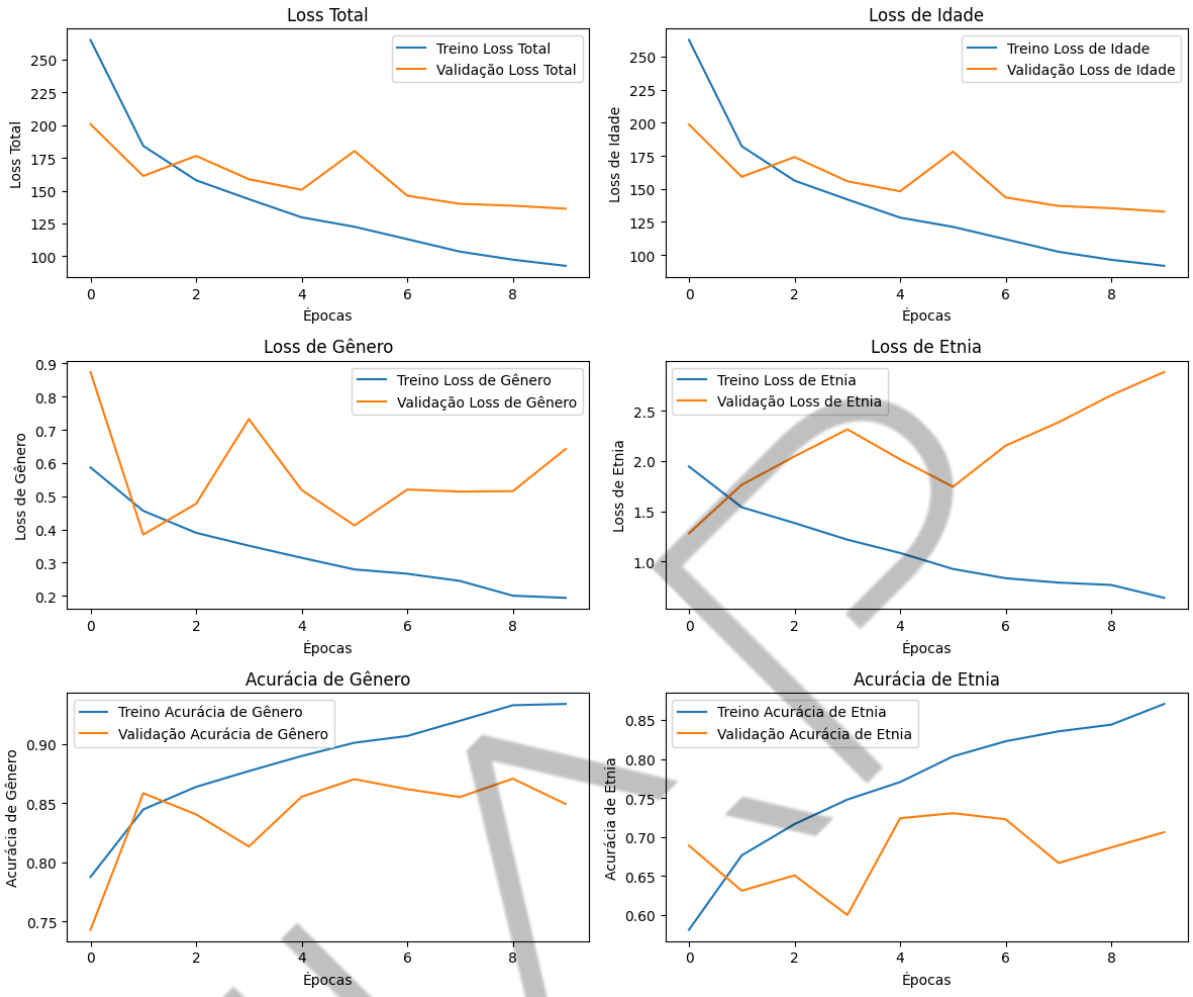

Figura 23 – Curvas de aprendizado por época
Fonte: autor (2024)

Pelos resultados, é possível notar que o modelo conseguiu aprender, apesar de apresentar sinais de sobreajuste. Isso se deve ao fato de a arquitetura da rede ser muito simples. Para otimizar o treinamento, podemos modificar a arquitetura da rede base, utilizando arquiteturas mais robustas como VGG ou ResNet com a técnica de transfer learning. Além disso, também é possível aprimorar a arquitetura das especializações de cada tarefa no final da rede. Durante o treinamento, podemos aumentar o número de épocas e utilizar checkpoints para otimização.

É possível realizar outras análises a fim de verificar a qualidade do treinamento, mas vou deixar para você explorar essas opções. Por exemplo, você pode examinar a matriz de confusão para tarefas de classificação, calcular métricas adicionais como a precisão, recall e F1-score, ou até mesmo analisar o desempenho em diferentes

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

subconjuntos de dados para entender melhor onde o modelo está se saindo bem ou mal.

2.5.3 Detecção de emoções em tempo real

Vamos ver o desempenho do nosso modelo na pratica; para isso, vamos verificar as predições do nosso conjunto de teste.

```
import tensorflow as tf
import numpy as np

model = tf.keras.models.load_model('model.h5')


def load_and_preprocess_image(file_path):
    image = tf.io.read_file(file_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, [128, 128])  # Tamanho da imagem
conforme usado no treinamento
    image = image / 255.0
    return image

def make_prediction(model, df, index=0):
    # Obter o caminho da imagem e os rótulos verdadeiros
    image_path = df.iloc[index]['image']
    age_true = df.iloc[index]['age']
    gender_true = df.iloc[index]['gender']
    ethnicity_true = df.iloc[index]['ethnicity']

    # Carregar e preprocessar a imagem
    image = load_and_preprocess_image(image_path)
    image = tf.expand_dims(image, axis=0)  # Adicionar dimensão do
batch

    # Fazer a predição
    age_pred, gender_pred, ethnicity_pred = model.predict(image)
    predicted_age = age_pred[0][0]
    predicted_gender = 'Masculino' if gender_pred[0][0] < 0.5 else
'Feminino'
    predicted_ethnicity = np.argmax(ethnicity_pred[0])

    # Classes de etnia (ajuste conforme necessário)
    ethnicity_classes = ["Branco", "Negro", "Asiático", "Indiano",
"Outros"]
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    # Visualizar a imagem com predições
    plt.figure(figsize=(6, 6))
    plt.imshow(plt.imread(image_path))
    plt.axis('off')
    plt.title(f"Idade Verdadeira: {age_true} | Predição:
{predicted_age:.2f}\n"
              f"Gênero Verdadeiro: {'Masculino' if gender_true == 0
else 'Feminino'} | Predição: {predicted_gender}\n"
              f"Etnia Verdadeira: {ethnicity_classes[ethnicity_true]}
| Predição: {ethnicity_classes[predicted_ethnicity]}")
    plt.show()

# faz predições
numero_imagem = 90
make_prediction(model, test_df, index= numero_imagem)
```

Código-fonte 23 – Realiza predições
Fonte: autor (2024)

O resultado esperado é a visualização da imagem informando os valores reais e os preditos. Na variável `numero_imagem`, você pode modificar para validar outras imagens como mostra a figura a seguir. Além disso, podemos fazer diversas modificações nesse código para que ele consiga realizar predições em vídeos e não apenas imagens estáticas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 24 – Exemplos de predições do modelo treinado
Fonte: autor (2024)

Em resumo, nesse capítulo, aprendemos:

- Técnicas avançadas de detecção de pessoas;
- Técnicas de reconhecimento de pessoas;
- Técnicas avançadas de arquiteturas de redes neurais;
- Aplicações de deep learning para reconhecimento de emoções;
- Criação de uma arquitetura de detecção de idade, sexo e etnia.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3 HANDS ON: PRATICANDO O RECONHECIMENTO DE PESSOAS

3.1 Aquecimento

Neste capítulo, vimos diversas técnicas avançadas de deep learning e como são aplicadas em problemas de visão computacional para problemas de reconhecimento de pessoas. Antes de começar o projeto do capítulo, certifique-se de ter acesso ao Google Colab e que o ambiente está ativo para uso da GPU. Isso será particularmente útil na etapa de treinamento do modelo.

6.2 Seu desafio

Sua tarefa é criar um modelo de reconhecimento de pessoas em tempo real que seja capaz de predizer os seguintes atributos:

- Emoções;
- Nome da pessoa;
- Sexo;
- Etnia;
- Idade.

Você deve:

- Aplicar as técnicas de pré-processamento e aumento de dados;
- Definir a arquitetura usando o TensorFlow, pode usar transfer learning;
- Treinar os modelos usando GPU;
- Carregar os modelos em um script que roda local do computador e rodar com webcam.

É esperado que o script seja capaz de informar com precisão os atributos solicitados na própria tela, como um output visual.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

6.3 Dicas de solução

A proposta é que você aplique os conceitos aprendidos sobre reconhecimento facial de maneira prática, construindo um sistema robusto que demonstre sua capacidade de funcionamento em tempo real.

Para a etapa de detecção de faces, você pode usar tanto Haar Cascade como Yunet, sendo o segundo mais preciso e superior ao primeiro. O Model Zoo te ajuda nessa parte.

Com a face detectada a próxima etapa é elaborar uma rede para reconhecimento de emoções, a VGGFace atende muito bem esse critério e pode ser utilizada.

Para o reconhecimento do nome da pessoa, não esquecer de adicionar uma imagem de referência para o cálculo de similaridade.

Para as tarefas de detecção de idade, sexo e etnia pode ser realizado um treinamento multitask com 3 saídas independentes.

Com as redes já treinadas e configuradas, crie um script python com opencv que carrega os modelos e realiza as predições de forma coerente.

Não é aconselhável o treinamento de uma única rede, pois não temos um dataset rotulado com todas essas informações.

Bons estudos!

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

ANWAR, Taha (2020). **Classification with localization**. Disponível em: <https://learnopencv.com/classification-with-localization//>. Acesso em: 14 abr. 2024.

ESCOLA DE ESTUDANTES. **Age and Gender Prediction with VGG-Face**. GitHub, 2020. Disponível em: <https://github.com/escoladeestudantes/keras/blob/master/10_Age_and_Gender_Prediction_VGG-Face_Sefik_Serengil/Keras_10_Age_and_Gender_Prediction_VGG-Face_Sefik_Serengil_01_image.py>. Acesso em: 2 jun. 2024.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep learning**. MIT press, 2016.

HAYKIN, Simon. **Redes neurais:** princípios e prática. Bookman Editora, 2008.

LANDUZGUN, Ilas. **Gender and age detection using with keras tensorflow image processing**. Medium, 2018. Disponível em: <https://medium.com/@ilaslanduzgun/gender-and-age-detection-using-with-keras-tensorflow-image-processing-90d7804473f8>. Acesso em: 2 jun. 2024.

LUGER, George F. **Inteligência Artificial**, 6ª ed. São Paulo: Pearson Education do Brasil, 2013 (biblioteca virtual).

MISHRA, Ranjan Kumar; REDDY, GY Sandesh; PATHAK, Himanshu. **The understanding of deep learning:** A comprehensive review. Mathematical Problems in Engineering, 2021, 2021: 1-15.

MITCHELL; Tom M. **Machine Learning**. 1st edition, New York, United States: McGraw-Hill Education, 1997.

RUSSEL, Stewart; NORVIG Peter**. Inteligência artificial**. 3ª. Ed., Rio de Janeiro: Campus, 2012.

SEFIKS, Sefik. **Facial expression recognition with Keras**. Sefiks Blog, 2018. Disponível em: https://sefiks.com/2018/01/01/facial-expression-recognition-with-keras/. Acesso em: 2 jun. 2024.

WANI, M. Arif; KHOSHGOFTAAR, Taghi M.; PALADE, Vasile (ed.). **Deep Learning Applications**, Volume 3. Springer, 2021.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com